QViz Documentation

Qubit Overflow

Contents

Preface . . . 11

Documentation overview . . . 13
Who this documentation is for . . . 13
How the book is organised . . . 13
Conventions used in this book . . . 14
A note on the word “backend” . . . 14

I Introduction

1 Why QViz . . . 17
What the visualiser allows . . . 17
Who it is for . . . 18
How to read the rest of the book . . . 18

II Getting started

2 Installation . . . 21
Requirements . . . 21
The versions that were tested . . . 21
What Qollab installs . . . 22
Installing the library . . . 22
Checking the installation . . . 22
Building this book . . . 22

3 Quickstart . . . 25
The two public functions . . . 25
A complete example . . . 25
The entry point for Qollab . . . 26
Measuring a step . . . 26
What happens behind the two calls . . . 26

4 Tests and demos . . . 29
Running the tests . . . 29
Running the linters . . . 29
Running a demo . . . 29
Generating all the example files . . . 29
Viewing the result . . . 30

III Core concepts

5 Pipeline overview . . . 33
The stages . . . 33
What each stage does . . . 35
Two rules that concern every module . . . 35
The sampler is outside the pipeline . . . 36
Where the data types are defined . . . 36

6 Units and conventions . . . 37
Angles . . . 37
The order of the qubits . . . 37
The Bloch vector . . . 37
Probabilities . . . 38
Fidelities . . . 38
Rounding . . . 38
Omitted fields . . . 38

7 JSON contract . . . 39
The top-level object . . . 39
The step object . . . 39
A full example . . . 40
Naming rules of the contract . . . 41

IV Architecture

8 System architecture . . . 45
The four parts . . . 45
How the parts are connected . . . 45
Three ways to run . . . 46
A request from end to end . . . 46
Where each rule is enforced . . . 47
What is not connected . . . 47

9 Project structure . . . 49
The repository . . . 49
Why `source/` and `resources/` are separate . . . 50
The modules of the library . . . 50
Module catalogue . . . 50
Who may import whom . . . 52
Where the tests are . . . 53

10 Public API and entry points . . . 55
The public API . . . 55

`qollab_api.py` . . . 55
`main.py` . . . 56
`resources/scripts/server.py` . . . 56
The other tools . . . 57
Choosing the right entry point . . . 57

11 **Engineering practices** . . . 59
Design . . . 59
One responsibility per module . . . 59
Modules do not import each other . . . 59
Orchestration is separate from logic . . . 59
Immutable data, pure functions . . . 59
Make illegal states hard to express . . . 59
Constants and configuration are different things . . . 60
Correctness . . . 60
Test against an independent oracle . . . 60
Tolerances are honest . . . 60
Validate at the boundary . . . 60
Fail with a precise error . . . 60
Never fail where classification is enough . . . 60
Safety and cost . . . 61
Maintainability . . . 61
Logging, not printing . . . 61
Write the specification first . . . 61
One module at a time, with a stop . . . 61
Document where the knowledge is lost . . . 61
Units are stated, always . . . 61
Delivery . . . 62
Practices in the viewer . . . 62
Known limits . . . 62

V **Backend foundations**

12 **Configuration, constants and errors** . . . 65
The two constant modules . . . 65
The fixed constants (`constants.py`) . . . 65
The tunable parameters (`config.py`) . . . 65
Which backends are accepted . . . 66
How the width limit is used . . . 66
The error module . . . 67
An unsupported provider is not an error of the detection . . . 67

13 **Data model** . . . 69
Type aliases . . . 69
`NoiseProfile` . . . 70
`BackendInfo` . . . 70
`TranspiledCircuit` . . . 70

`NativeOp` . . . 71
`NativeCircuit` . . . 71
`QubitAngles` . . . 72
`SimulatedStep` and `SimulatedCircuit` . . . 72
`VectorizedStep` and `VectorizedCircuit` . . . 72
`MeasuredStep` and `MeasuredCircuit` . . . 73

14 Provider . . . 75
`detect(backend, entangler=None)` . . . 75
How the entangler is chosen . . . 75
The IonQ machine family . . . 75
How the hardware flag is set . . . 76
`detect_provider(backend)` . . . 76
The first signals . . . 76
The structural fallback . . . 76
The noise profiles . . . 77
`profile_for(provider, name, entangler)` . . . 77

VI Backend input and translation

15 Importer . . . 81
Why it is needed . . . 81
`load_circuit(text)` . . . 81
Loading without spending the IonQ job . . . 82
In Qollab . . . 82

16 Transpiler . . . 83
`transpile(info, backend, qc, *, allow_remote=True)` . . . 83
What is done first . . . 83
The IonQ strategies . . . 84
The local simulator . . . 84
The layout . . . 85
The meaning of the `source` field . . . 85

17 Translator . . . 87
`translate(info, transpiled)` . . . 87
The translation rules . . . 87
`emit(ops, op, qubits, entangler, source_index=None)` . . . 88
`emit_zz(ops, targets, angle, entangler, source_index=None)` . . . 89
`emit_ms(ops, targets, phases, angle, entangler, source_index=None)` . . . 89
`translate_circuit(circuit, layout, entangler)` . . . 89
`translate_native_ops(native_ops, num_qubits, entangler)` . . . 89
The compaction of the qubits . . . 90
The normalization of the angle . . . 90
`normalize_zz(targets, angle)` . . . 90

`normalize_ms(targets, phases, angle)` . . . 91
A note on global phases . . . 91

18 The `to_qiskit` bridge . . . 93
Why it is not inside the translator . . . 93
`to_qiskit(native, upto=None)` . . . 93
`circuit_from_ops(ops, num_qubits)` . . . 93

VII Backend simulation and output

19 Simulator . . . 97
`simulate(native)` . . . 97
How the state is stored . . . 97
How a qubit is described . . . 97
The `virtual_z` operation . . . 97
The gate matrices . . . 98
`gpi.matrix(phase)` . . . 98
`gpi2.matrix(phase)` . . . 98
`ms.matrix(phase0, phase1, angle)` . . . 98
`zz.matrix(angle)` . . . 98
`virtualz.matrix(rotation)` . . . 98

20 Vectorizer . . . 99
`vectorize(simulated)` . . . 99
The conversion . . . 99

21 Metrics . . . 101
`measure(vectorized, info, shots=1024)` . . . 101
The initial fidelity . . . 101
The per-step rules . . . 101
What a `virtual_z` does not change . . . 102
The ideal profile . . . 102
The guarantees . . . 102

22 Formatter and exporter . . . 103
`format(measured)` . . . 103
What the function does . . . 103
`export(data, path=None)` . . . 103
Why the dictionary and the string are separated . . . 104

VIII Backend orchestration and extras

23 Compiler . . . 107
`compile(backend, qc, *, entangler=None, strategy="statevector", shots=1024, allow_paid=False, allow_remote=True)` . . . 107

The steps that are performed . . . 108
A complete example . . . 108

24 Sampler . . . 111
Why it is needed . . . 111
`sample(data, step, shots=1024, *, seed=None)` . . . 111
How the counts are computed . . . 112
The noise model . . . 112
`probabilities(data, step)` . . . 112
The Qollab functions . . . 112
The measurement of the viewer . . . 112

25 Tomography . . . 115
Why the state is measured, and not computed . . . 115
`measure(native, backend, info, *, shots=1024, allow_paid=False)` . . . 115
Why the circuit is cut into prefixes . . . 115
Why three bases are needed . . . 116
Why all the circuits are placed in one call . . . 116
How an expectation is found . . . 116
Known limitations and conflicts . . . 116

IX Examples

26 The QFT adder . . . 121
The quantum Fourier transform . . . 121
The Draper adder . . . 121
The smallest adder . . . 122
Running the example . . . 122
A note on the number of steps . . . 122

27 Performance circuits . . . 123
`ghz(count)` . . . 123
`layered(count, depth=4, seed=1234)` . . . 123
Generating the files . . . 123
Opening the files . . . 123

X Frontend

28 Viewer architecture . . . 127
Layers . . . 127
Shared state . . . 127
From a file to the screen . . . 128
Coordinates . . . 129

The shaders . . . 129
Layers of the scene . . . 129
The host . . . 130
Testing without a browser . . . 130

29 Basic viewer . . . 131
The two-qubit gate . . . 132
The surface-trap chip . . . 133
The horizontal layer . . . 133
Preparing the ions (the intro) . . . 133
The laboratory environment . . . 133
The interface . . . 134
What is shown . . . 134
How a file is loaded . . . 135
The controls . . . 135
The View and accessibility panel . . . 136
The sound of the gates . . . 137
The circuit timeline . . . 138
When a circuit cannot be shown . . . 138
The measurement . . . 139
How the animation works . . . 139
The fields that are read . . . 139
The files of the viewer . . . 139

30 The viewer on Qollab . . . 143
At a glance . . . 143
The rule . . . 143
What Qollab does and does not allow . . . 143
How it is built . . . 144
The prelude of `main.js` . . . 145
The hooks of the viewer . . . 145
The measurement . . . 146
`qollab_api.py` . . . 146
Problems found on Qollab and how they were solved . . . 146
How it was verified . . . 147
Rules for future changes . . . 147

XI Development

31 Tests . . . 151
The shared circuits . . . 151
The fixtures . . . 151
The test files . . . 152

The oracles . . . 153
The golden file . . . 153
Numerical tolerances . . . 153
Continuous integration . . . 154

32 Demos . . . 155
The shared helpers . . . 155
demo_transpiler.py . . . 155
demo_translator.py . . . 155
demo_simulator.py . . . 155
32.1 demo_vectorizer.py . . . 155
demo_metrics.py . . . 156
demo_compile.py . . . 156
demo_adder.py . . . 156
package_qollab.py . . . 156
generate_outputs.py . . . 156

33 Tooling and workflow . . . 157
The Makefile . . . 157
Setting up and checking . . . 157
Producing and serving . . . 157
Diagrams . . . 158
The book . . . 158
The scripts . . . 158
Continuous integration . . . 159
Branches and commits . . . 159
How a module is built . . . 159
Working with code agents . . . 160
The tests of the project itself . . . 160
Before closing a change . . . 160

34 Style and conventions . . . 161
The layout of the project . . . 161
One module per package . . . 161
Imports . . . 161
Purity . . . 162
No printing inside the library . . . 162
Comments and docstrings . . . 162
Type hints . . . 162
Line length and formatting . . . 162
The theme of the book . . . 162
The memory of the project . . . 163
Git . . . 163

Preface

QViz is an educational visualiser for trapped-ion quantum circuits. A Qiskit circuit and a chosen backend are turned into a JSON file. That file describes the native ion gates and the state of every qubit after every gate. A web page then draws the ions as Bloch spheres.

The purpose is **pedagogical**. There is a gap between the logical gates of a circuit and the native gates that the machine really runs. By showing every native gate and the state of every ion, the gap is closed, and the physics of each gate can be explained. See Why QViz.

This book documents the whole project. Its main subject is the **backend** (Python), which is described module by module. It also covers how the parts are connected, the **viewer** that draws the JSON, and its use on Qollab.

The book is written in plain English. Short sentences and the passive voice are used, so that the reader is not required to be an expert. Every public function is described, together with its parameters, its return value and its units.

The pages are grouped as follows.

- **Introduction** explains why the project exists.
- **Getting started** shows how the project is installed and run.
- **Core concepts** explains the pipeline, the units and the JSON contract.
- **Architecture** shows how the library, the adapters, the viewer and the hosts are connected, how the repository is organised and which engineering practices are followed.
- **Backend foundations** documents the constants, the errors, the data model and the provider.
- **Backend input and translation** documents the importer, the transpiler, the translator and the `to_qiskit` bridge.
- **Backend simulation and output** documents the simulator, the vectorizer, the metrics and the formatter and exporter.
- **Backend orchestration and extras** documents the compiler, the sampler and the tomography.
- **Examples** shows complete circuits, such as the QFT adder.
- **Frontend** documents the architecture of the viewer, the viewer itself and its use on Qollab.
- **Development** explains the tests, the demo scripts, the style rules and the tooling and workflow.

The complete rendered book can be built with Quarto. See Installation for the commands.

Documentation overview

This page explains how the documentation is organised, and which conventions are used everywhere in the book.

Who this documentation is for

The book has two audiences.

- For the **project**: a new contributor, who needs to understand the code before it is changed; a reviewer, who needs to check that the backend behaves as specified; and a teammate, who needs to know how the JSON is produced, so that a visualiser can be connected to it.
- For the **teaching**: a teacher or a student, who wants to see what a quantum circuit really does on a trapped-ion machine. It is the reason the project exists; see Why QViz.

No previous knowledge of trapped-ion hardware is assumed. The few physics terms that are needed are defined where they first appear.

How the book is organised

Part	What is found there
Introduction	Why the project exists, and the pedagogical idea behind it.
Getting started	Installation, a first example and the test and demo commands.
Core concepts	The pipeline, the units and the JSON contract.
Architecture	How the parts connect, the structure of the repository, the entry points and the engineering practices.
Backend foundations	The constants, the errors, the data model and the provider. One page per module, with every function and parameter.
Backend input and translation	The importer, the transpiler, the translator and the `to_qiskit` bridge.
Backend simulation and output	The simulator, the vectorizer, the metrics and the formatter and exporter.
Backend orchestration and extras	The compiler, the sampler and the tomography.
Examples	Complete circuits, such as the QFT adder and the performance tests.
Frontend	The architecture of the viewer, the viewer itself and its use on Qollab.

Part	What is found there
Development	Tests, demos, the coding style, the tooling and the workflow.

Conventions used in this book

- **Passive voice.** The text is written in the passive voice where it is natural, for example “the circuit is transpiled” rather than “we transpile the circuit”.
- **Simple sentences.** One idea is given per sentence. Long sentences are avoided.
- **Units are always stated.** An angle is never given without saying whether it is measured in radians or in turns. The units are collected on the Units and conventions page.
- **Code is shown, not executed.** Every code block is written in plain `python` (or `json`) form. Nothing is run while the book is rendered, so the book can be built without installing the package.
- **Identifiers are kept in English.** Function names, parameter names and file names are written exactly as they appear in the code.

A note on the word “backend”

In Qollab, the platform where the project runs, the word *backend* is used in two different ways.

1. The **library**: the Python package named `backend` that contains the pipeline. This is documented in the book.
2. The **backend object**: the quantum backend that the platform injects into `main.py` (for example an IonQ simulator). It is received as an argument and is never imported.

The meaning is clear from the context on each page.

I Introduction

1 Why QViz 17
1 What the visualiser allows 17
1 Who it is for 18
1 How to read the rest of the book 18

1. Why QViz

A quantum program is written with logical gates: `h`, `cx`, `rz`, and so on. But a trapped-ion machine does not run those gates. It runs a small set of **native** gates: GPI, GPI2, a two-qubit entangler (MS or ZZ), and a virtual Z rotation. The logical circuit is transpiled and translated until only those gates remain.

The two pictures are very different. A single `cx` becomes a short sequence of native gates, and each of them turns the state of an ion in a particular way. This gap between the abstract circuit and the real machine is where the understanding is often lost.

QViz makes the gap visible. The circuit is compiled into its native gates, and the state of every ion is computed after **every** gate. A web page then draws each ion as a Bloch sphere, step by step.

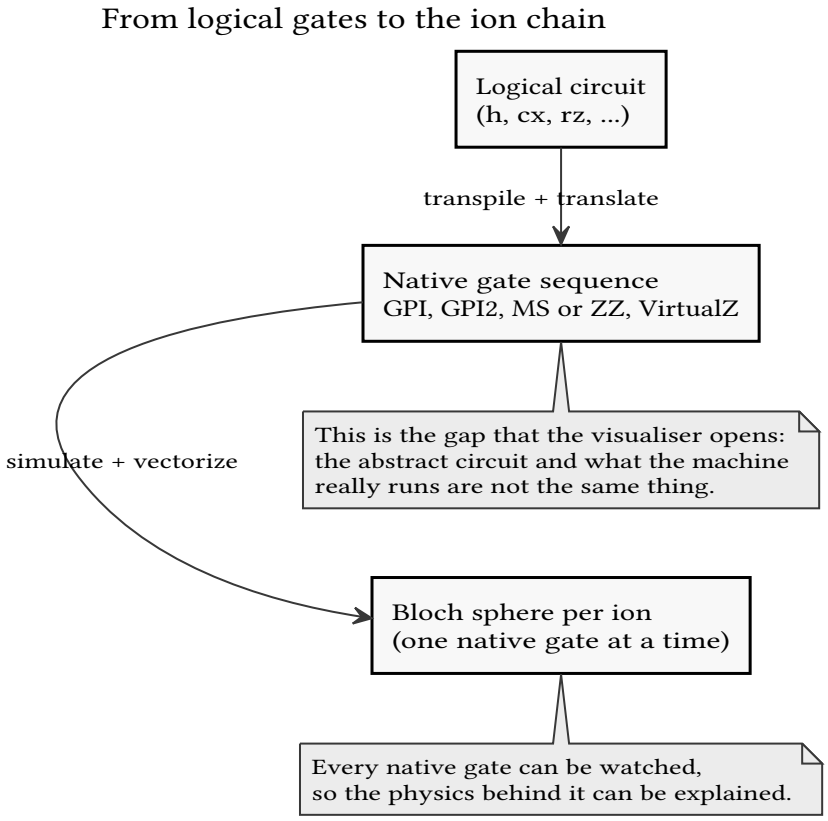


Figure 1.1: From logical gates to the ion chain

What the visualiser allows

- To see how a logical gate is built from native gates.
- To see what one native gate does to one ion, and to the chain.
- To see entanglement appear: the Bloch vector of an ion shrinks, and disappears when the ion is fully entangled.
- To see the noise: the health of the circuit falls with every gate, and a noisier ion moves a little more.
- To explain the physics of each gate, because the state before and after the gate is on the screen.

Who it is for

It is written for a general public and for teaching. No previous knowledge of trapped-ion hardware is assumed. The interface is bilingual (Spanish and English), so that it can be used in a classroom without a translation step.

How to read the rest of the book

- Getting started installs the project and shows a first example.
- Core concepts explains the pipeline, the units and the JSON contract.
- Backend foundations, input and translation, simulation and output and orchestration and extras document each module.
- Examples shows complete circuits, such as the QFT adder and the 16-qubit performance circuits.
- Frontend documents the viewer.

II Getting started

2 **Installation** 21
2 Requirements 21
2 Installing the library 22
2 Checking the installation 22
2 Building this book 22

3 **Quickstart** 25
3 The two public functions 25
3 A complete example 25
3 The entry point for Qollab 26
3 Measuring a step 26
3 What happens behind the two calls 26

4 **Tests and demos** 29
4 Running the tests 29
4 Running the linters 29
4 Running a demo 29
4 Generating all the example files 29
4 Viewing the result 30

2. Installation

This page describes how the project is installed on a local computer, and how the documentation is built.

Requirements

Software	Version	Why it is needed
Python	3.12 or newer	The backend is written in Python. Qollab uses 3.14.
Qiskit	`>=2.5.2,<3.0.0`	Circuits, transpilation and the test oracles. This is the line that Qollab adds by itself.
qiskit-ionq	`>=1.0.0,<2.0.0`	The IonQ native gates and the IonQ backend.
NumPy	`>=2.0,<3.0`	The statevector and the gate matrices.
qiskit-qasm3-import	`>=0.6,<1.0`	Reads OpenQASM 3. It is pure Python; OpenQASM 2 works without it.
Quarto	1.4 or newer	Only for building this book.

The Python packages are declared in `pyproject.toml`. The same packages, without the development tools, are declared in `requirements.txt` for Qollab, and a test checks that the two lists are equal.

The versions that were tested

Qollab ships its own `qiskit-ionq`, and the version is not documented. The library does not use a function that only the newest versions have, so any version from 1.0.0 works. The whole test suite was run with each of the following.

qiskit-ionq	Result
1.0.0, 1.0.1, 1.0.2	Everything works, except the tomography on a backend that is not IonQ. The native gates of these versions have no definition, so Qiskit cannot transpile them. The tomography then raises a `TranspilationError` that says so.
1.0.3, 1.1.0, 1.1.1	Everything works.

Qiskit 2.5.2 and NumPy 2.4.6 (and NumPy 2.0.2 with `qiskit-ionq` 1.0.0) were used.

The latest full run (2026-10-11) used Python 3.13.16, Qiskit 2.5.2, qiskit-ionq 1.1.1, NumPy 2.5.4 and qiskit-qasm3-import 0.6.0: 419 tests passed, 1 was skipped, and `make lint` had no findings. A machine whose `python3` is older than 3.12 (for example Debian 12, with 3.11) cannot install the package: `pip` refuses it with `requires a different Python`. Create the environment with a newer interpreter, for example `uv venv --python 3.13 .venv`.

The viewer loads three.js from a CDN and does not install it, so it is not in `requirements.txt` (that file is for Python). On Qollab the platform provides three.js 0.186.1, and the import map of `source/frontend/index.html` pins the same version, so the standalone viewer looks like the one on Qollab. Until 2026-10-11 it pinned 0.160.0, and the scene looked much brighter on Qollab: three.js 0.163 and later light the scene with the environment map more strongly. The scene is calibrated for 0.186.1 with `ENVIRONMENT_INTENSITY` in `source/frontend/js/config.js`. Qollab runs Python 3.14 (Pyodide v314.0.7). On Qollab, the page of the test project prints the versions that are really installed (see `documentation/prompts/qollab_pruebas.md`).

What Qollab installs

The rules of the platform for `requirements.txt` are the following (see the runtime environment page of Qollab).

- One package per line, as `name==1.2.3` or as a range. Comments and blank lines are allowed.
- Pip options, environment markers and URL requirements are rejected.
- Qiskit is always installed. When the file does not name it, Qollab adds `qiskit>=2.5.2,<3.0.0`; when it does, the line is used as written. The line of this project is the same as the default.
- The packages that Pyodide has built (NumPy, SciPy, Matplotlib) load when they are imported. A pure-Python wheel from PyPI is installed.
- A package that needs compiled code, and has no Pyodide build, stops the run with `no installable distribution for '<name>'`.
- Qollab has no `qiskit-ionq` in its documentation, but the IonQ backends are built with it. This is why its range does not ask for a newer version than the platform has: a range that the installed version satisfies leaves it as it is.

Installing the library

A virtual environment is created and the package is installed in editable mode. The commands are run from the project root.

```
make venv          # creates the .venv directory
make install       # installs the package with its development extras
```

The `make install` command runs `pip install -e ".[dev]"`. The development extras are `pytest`, `jsonschema`, `flake8` and `pydocstyle`.

The same steps can be done by hand:

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -e ".[dev]"
```

Checking the installation

```
make test          # runs the test suite with pytest
make lint          # runs flake8 and pydocstyle
```

If both commands finish without an error, the installation is correct.

Building this book

The book is a Quarto project, configured by `documentation/_quarto.yml`. The Makefile includes the Quarto targets, so they are run from the repository root. The following commands are available.

```
make render       # builds the HTML book into documentation/_book/
make preview      # live preview on http://localhost:4321
make pdf-typst    # builds a PDF with Typst
make clean        # removes the build output
```

The output is written to the `documentation/_book` directory. This directory is not committed.

3. Quickstart

This page shows the smallest possible use of the backend. A two-qubit Bell circuit is compiled, and the resulting JSON is printed.

The two public functions

Only two functions are needed.

```
from backend import compile, export

data = compile(backend, circuit)   # a Python dictionary
text = export(data)                # a JSON string
```

- `compile` runs the whole pipeline and returns a dictionary.
- `export` turns that dictionary into a JSON string, and can also write it to a file.

The variable `backend` is provided by Qollab. On a local computer, a local simulator is accepted, for example `BasicSimulator()`. Only IonQ machines and local simulators are accepted; any other backend is rejected with `UnsupportedProviderError` (see the configuration page).

A complete example

```
from qiskit import QuantumCircuit
from qiskit.providers.basic_provider import BasicSimulator

from backend import compile, export


def build_circuit() -> QuantumCircuit:
    """Build a two-qubit Bell-state circuit."""
    circuit = QuantumCircuit(2)
    circuit.h(0)
    circuit.cx(0, 1)
    return circuit


backend = BasicSimulator()
data = compile(backend, build_circuit(), shots=1024)
print(export(data))
```

When the example is run, the JSON that is produced starts as follows. The first two steps are the translation of the Hadamard gate on qubit 0: a virtual Z rotation, followed by a `gpi2` rotation.

```
{
  "backend": "basic_simulator → iones (referencia Aria)",
  "total_qubits": 2,
  "initial_fidelity": 99.5,
  "steps": [
    {
      "step": 1,
      "operation": "virtual_z",
```

```
      "targets": [0],
      "cumulative_fidelity": 99.5,
      "states": [[0.0, 0.0, 1.0], [0.0, 0.0, 1.0]],
      "probabilities": [[1.0, 0.0], [1.0, 0.0]]
    }
  ]
}
```

Only a few fields are shown above. The full list of fields is given in the JSON contract page.

The entry point for Qollab

The file `main.py` in the project root is the entry point on Qollab. It compiles an example circuit, prints the JSON (or only its size, when the circuit has more than 30 steps), and then measures one step with `sample`. Its signature is strict, so Qollab shows a form in the *Select QPU* dialog with four fields. The functions are in `qollab_api.py`, which a JS page can call too.

Field	Default	Meaning
`circuit`	`bell`	`bell`, `bell_distant`, `ghz3`, `ghz16` or `layered16`.
`shots`	`1024`	The number of shots, for the cost and for the measurement. It must be positive.
`entangler`	`auto`	`auto` lets the backend choose: ZZ for a Forte machine, and MS for every other backend. `ms` and `zz` force the entangler. The case is ignored.
`step`	`0`	The step to measure. `0` means the last one.

The fields `circuit` and `entangler` are free text, because the form has no drop-down list. Any other text is rejected with an error that names the valid values, so that a typing mistake is never silently ignored.

Measuring a step

A third function, `sample`, simulates a measurement of any step of the result. The exact joint distribution is used, so the correlations of an entangled state are kept. See the Sampler page.

```
from backend import compile, sample

data = compile(backend, circuit)
counts = sample(data, step=len(data["steps"]), shots=1024)
# For a Bell pair: {'00': 513, '11': 496, '01': 8, '10': 7}
```

What happens behind the two calls

When `compile` is called, the following steps are performed, in this order.

1. The provider of the backend is detected.
2. The circuit is transpiled for that backend.
3. The transpiled circuit is translated into the native ion gates.

4. The state of every qubit is computed after every native gate.
5. The state is converted into Cartesian Bloch vectors.
6. The didactic metrics are added.
7. The result is formatted as a dictionary.

Each of these steps is documented on its own page. The overview is given on the Pipeline overview page.

4. Tests and demos

This page describes how the automated tests and the demo scripts are run.

Running the tests

```
make test
```

The command runs `pytest` over the `source/tests/` directory. All the tests must pass before a module is considered finished.

A single test file can also be run.

```
.venv/bin/python -m pytest source/tests/test_translator.py -v
```

The tests are described in detail on the Tests page.

Running the linters

```
make lint
```

Two tools are run.

- `flake8` checks the PEP 8 style, with a maximum line length of 88 characters.
- `pydocstyle` checks that the docstrings follow PEP 257.

Running a demo

Each pipeline stage has a small script that prints a readable result.

```
make demo M=transpiler
make demo M=translator
make demo M=simulator
make demo M=vectorizer
make demo M=metrics
make demo M=compile
make demo M=adder
```

The `M` variable is required. The command runs `resources/scripts/demo_<module>.py` with the virtual environment.

The `demo_compile` script writes two files, `out/bell.json` and `out/ghz3.json`. The `demo_adder` script writes the adder files. The `out` directory is ignored by Git. These files can be opened with the 3D viewer.

Generating all the example files

```
make outputs
```

The script compiles a small set of circuits, including two 16-qubit performance circuits, and writes them to `out/`. It also writes `out/manifest.json`, which is read by the viewer. See the Performance circuits page.

Viewing the result

The JSON that is produced can be inspected by hand, or with the viewer.

```
make outputs
make server
# then open http://localhost:8000/source/frontend/index.html
```

The page `source/frontend/index.html` is connected to the files in `out/`. It reads the manifest, fills a selector with the available files, and opens the default one. An OpenQASM file (`resources/examples/qasm/*.qasm`) can be loaded with the button *Cargar circuito* or dragged onto the page: `make server` compiles it (a plain `python -m http.server` cannot).

III Core concepts

5 **Pipeline overview** 33
5 The stages 33
5 What each stage does 35
5 Two rules that concern every module 35
5 The sampler is outside the pipeline 36
5 Where the data types are defined 36

6 **Units and conventions** 37
6 Angles 37
6 The order of the qubits 37
6 The Bloch vector 37
6 Probabilities 38
6 Fidelities 38
6 Rounding 38
6 Omitted fields 38

7 **JSON contract** 39
7 The top-level object 39
7 The step object 39
7 A full example 40
7 Naming rules of the contract 41

5. Pipeline overview

The backend is built as a pipeline. Each stage receives one object, and returns a new object of a different type. The stages are connected by the `compiler` module, and by nothing else.

The stages

```
QuantumCircuit + backend
        |
        ▼
[provider]   detect(backend)              ⟶ BackendInfo
        |
        ▼
[transpiler] transpile(info, backend, qc) ⟶ TranspiledCircuit
        |
        ▼
[translator] translator.translate(...)    ⟶ NativeCircuit
        |
        ▼
[simulator]  simulate(native)             ⟶ SimulatedCircuit
        |
        ▼
[vectorizer] vectorize(simulated)         ⟶ VectorizedCircuit
        |
        ▼
[metrics]    measure(vectorized, info)    ⟶ MeasuredCircuit
        |
        ▼
[formatter]  format(measured)             ⟶ dict
        |
        ▼
[exporter]   export(data)                 ⟶ str (JSON)
```

The same order is shown in the activity diagram.

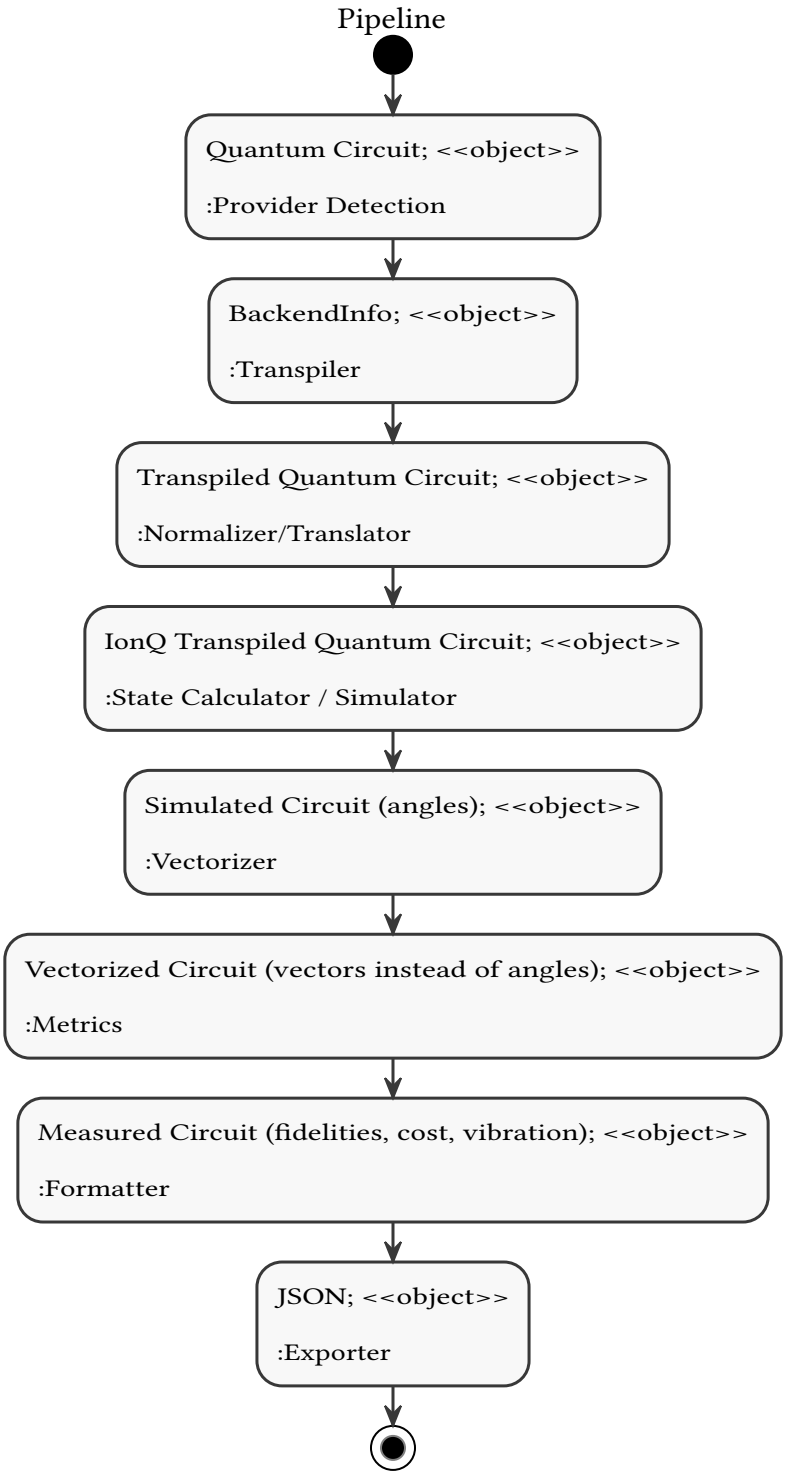


Figure 5.1: The pipeline: from the circuit to the JSON.

The modules that implement the stages, and the way they depend on each other, are shown next. Only `compiler` knows every module.

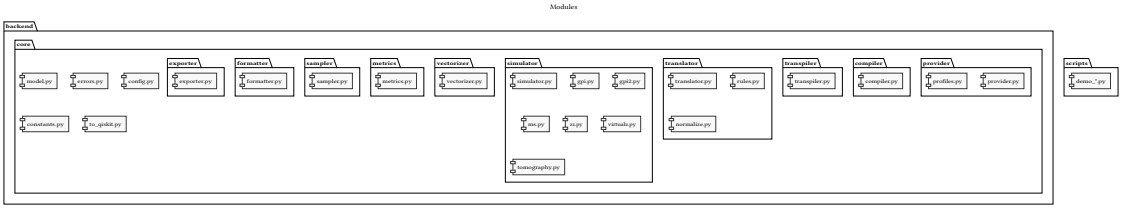


Figure 5.2: The modules of the backend.

What each stage does

Stage	Input	Output	Short description
provider	a backend object	`BackendInfo`	The provider, the entangler and the noise profile are found.
transpiler	circuit and backend	`TranspiledCircuit`	The circuit is changed into the gates of the backend.
translator	`TranspiledCircuit`	`NativeCircuit`	Only `gpi`, `gpi2`, `ms` or `zz`, and `virtual_z` are kept.
simulator	`NativeCircuit`	`SimulatedCircuit`	The state of each qubit is computed after each gate.
vectorizer	`SimulatedCircuit`	`VectorizedCircuit`	Angles are changed into `(x, y, z)` vectors.
metrics	`VectorizedCircuit`	`MeasuredCircuit`	Fidelities, inactivity, vibration and cost are added.
formatter	`MeasuredCircuit`	`dict`	The dictionary of the JSON contract is built.
exporter	`dict`	`str`	The dictionary is written as JSON.

Two rules that concern every module

Two rules are respected by all the modules of the pipeline.

1. **No module imports another pipeline module.** A module may import only `core.model`, `core.errors`, `core.config` and, when needed, `core.provider` and `core.to_qiskit` (the neutral bridge from a native circuit to a Qiskit circuit, used by the tomography and by the tests). Only `compiler` knows all the other modules. This keeps the modules easy to test and to change.
2. **The public functions are pure.** An object is received, and a new object is returned. The input is never changed. There is no global state. There are no `print` calls inside `source/backend/core`; the `logging` module is used instead.

There is one documented exception. The `transpiler` and `tomography` modules are allowed to call `backend.run`, because a real measurement is needed. The rule of “one job per run” is respected.

The sampler is outside the pipeline

The `sampler` is not a stage. It works on the dictionary that the pipeline returns, to simulate a measurement of any step, and it is exported as `backend.sample`. Like the stages, it imports only `core.model`, `core.errors`, `core.config` and `core.to_qiskit`. See the Sampler page.

Where the data types are defined

Every object that travels between the stages is defined as a frozen dataclass in `source/backend/core/model.py`. A frozen dataclass cannot be changed after it is created, so the purity rule is easy to keep. The types are documented on the Data model page.

6. Units and conventions

The same units are used everywhere in the backend. They are collected on this page, because a wrong unit is the most common source of error.

Angles

Quantity	Unit	Notes
Phase of `gpi`, `gpi2` and `ms`	radians	In `qiskit-ionq` a phase is given in turns. The conversion is `radians = 2 * pi * turns`.
`angle` of `ms` and `zz`	turns, in the range [0, 0.25]	`0.25` is the maximum entanglement.
Rotation of `virtual_z`	radians	It is kept only in the intermediate representation. It is not written in the JSON for each step; the accumulated value is written instead.

The conversion from turns to radians is always made at the border of the library. Inside the backend, radians are used for the phases. A turn is equal to `2 * pi` radians. The constant `TWO_PI` is defined in `constants.py`.

The order of the qubits

The Qiskit convention is used. It is called **little-endian**.

- Qubit 0 is the least significant bit.
- In a bitstring, the rightmost character is qubit 0.

For example, the bitstring `"01"` means that qubit 0 is `1` and qubit 1 is `0`.

In the simulator, the state is stored as a tensor of shape `(2,) * n`. Qubit `q` lives on the axis `n - 1 - q`. With this choice, the natural order of the tensor is also little-endian.

The Bloch vector

The state of a single qubit is described by a vector on the Bloch sphere.

State	Vector `(x, y, z)`
`|0⟩`	`(0, 0, 1)`
`|1⟩`	`(0, 0, -1)`
state on the equator	`z = 0`

The vector is defined by the expectation values of the Pauli operators.

$$x = \langle X \rangle, \qquad y = \langle Y \rangle, \qquad z = \langle Z \rangle \tag{6.1}$$

The length of the vector is called `r`. If the qubit is entangled with another qubit, then `r` is smaller than 1. The length is never larger than 1.

The vector is also described by two angles.

Angle	Range	Meaning
`theta`	$[0, \pi]$	The angle from the `+z` axis.
`phi`	$[0, 2\pi)$	The angle from the `+x` axis, in the `xy` plane.

Two special cases are used. If `r` is close to zero, then `theta` and `phi` are both set to zero. If `theta` is close to 0 or to π, then `phi` is set to zero.

The standard physics convention is always used by the backend. If the frontend wants to draw the south pole upwards, that is the responsibility of the frontend.

Probabilities

For each qubit, two probabilities are given.

$$p(|0\rangle) = \frac{1+z}{2}, \qquad p(|1\rangle) = \frac{1-z}{2} \tag{6.2}$$

The two values always add up to one.

Fidelities

Fidelities are given as a **percentage**, in the range $[0, 100]$. A percentage is used because the initial fidelity is defined as `100 - SPAM`, and SPAM is a percentage-like error.

Rounding

All floating-point numbers are rounded to `JSON_DECIMALS` decimals, which is 6 by default. The negative zero `-0.0` is written as `0.0`.

Omitted fields

A field that does not apply to an operation is **left out** of the JSON. It is not written as `null`. For example, a `virtual_z` step has no `phases` field, and a `zz` step has no `phases` field either.

7. JSON contract

The JSON file is the only connection between the backend and the frontend. The format is fixed, and it is not changed without an agreement of the team. The schema is stored in `documentation/schema/visualization.schema.json`.

The top-level object

Key	Type	Meaning
`backend`	string	The readable name of the backend. For a non-IonQ backend, the name of the reference ion machine is added.
`total_qubits`	integer	The number of qubits after compaction, from 1 to 16 (`MAX_QUBITS`). Unused physical qubits are removed.
`initial_fidelity`	number	`100 * (1 - spam_error)`, in percent.
`steps`	array	One object per native operation, ordered by `step`.

The step object

Key	Type	Present for	Meaning
`step`	integer	always	The index of the step. It starts at 1 and it is consecutive.
`operation`	string	always	One of `gpi`, `gpi2`, `ms`, `zz`, `virtual_z`. Lower case is always used.
`targets`	array of integers	always	The qubits that the operation acts on. One qubit for `gpi`, `gpi2` and `virtual_z`. Two qubits for `ms` and `zz`.
`phases`	array of numbers	`gpi`, `gpi2`, `ms`	The logical phases, in radians. One value for `gpi` and `gpi2`. Two values for `ms`.

Key	Type	Present for	Meaning
`angle`	number	`ms`, `zz`	The entanglement angle, in turns, in the range [0, 0.25].
`cumulative_fidelity`	number	always	The global fidelity after the step, in percent.
`states`	array of `[x, y, z]`	always	The reduced Bloch vector of every qubit, after the step. One entry per qubit.
`vibration`	array of numbers	always	The vibration amplitude of every qubit, in [0, 1]. It is zero below the threshold.
`cumulative_cost`	number	always	The estimated cost so far, in USD, for the given number of shots.
`probabilities`	array of `[p0, p1]`	always	`[p(|0⟩), p(|1⟩)]` for every qubit.
`virtual_z`	array of numbers	always	The accumulated virtual Z rotation of every qubit, in radians, wrapped to [0, 2π).
`inactivity`	array of integers	always	The accumulated number of steps in which the qubit was not a target.
`fidelities`	array of numbers	always	The fidelity of every qubit after the step, in percent. It starts at 100.

The `phases` field is the **logical** phase, that is, the phase that appears in the circuit with explicit virtual Z gates. The physical laser phase would be `phase - virtual_z[qubit]`, and it is not written.

The lengths of `states`, `vibration`, `probabilities`, `virtual_z`, `inactivity` and `fidelities` are all equal to `total_qubits`.

A full example

```
{
  "backend": "IonQ Aria 1",
  "total_qubits": 2,
  "initial_fidelity": 99.5,
  "steps": [
    {
```

```
      "step": 1,
      "operation": "virtual_z",
      "targets": [0],
      "cumulative_fidelity": 99.5,
      "states": [[0.0, 0.0, 1.0], [0.0, 0.0, 1.0]],
      "vibration": [0.0, 0.0],
      "cumulative_cost": 0.0,
      "probabilities": [[1.0, 0.0], [1.0, 0.0]],
      "virtual_z": [3.141593, 0.0],
      "inactivity": [0, 0],
      "fidelities": [100.0, 100.0]
    },
    {
      "step": 2,
      "operation": "gpi2",
      "targets": [0],
      "phases": [1.570796],
      "cumulative_fidelity": 99.48,
      "states": [[1.0, 0.0, 0.0], [0.0, 0.0, 1.0]],
      "vibration": [0.0, 0.0],
      "cumulative_cost": 0.0003,
      "probabilities": [[0.5, 0.5], [1.0, 0.0]],
      "virtual_z": [3.141593, 0.0],
      "inactivity": [0, 1],
      "fidelities": [99.98, 99.99]
    }
  ]
}
```

Naming rules of the contract

The names in the contract follow two rules.

- The prefix `cumulative_` is used, as in `cumulative_fidelity` and `cumulative_cost`.
- The operation is written `virtual_z`, in snake case. It matches the `virtual_z` field, and no capital letter appears inside the enumeration.

If a name is changed, it must be changed in the backend, in this documentation and in the JSON schema at the same time, and before the frontend is written.

IV Architecture

8 System architecture 45
8 The four parts 45
8 How the parts are connected 45
8 A request from end to end 46
8 Where each rule is enforced 47
8 What is not connected 47

9 Project structure 49
9 The repository 49
9 Why `source/` and `resources/` are separate . . . 50
9 The modules of the library 50
9 Module catalogue 50
9 Who may import whom 52
9 Where the tests are 53

10 Public API and entry points 55
10 The public API 55
10 `qollab_api.py` 55
10 `main.py` 56
10 `resources/scripts/server.py` 56
10 The other tools 57
10 Choosing the right entry point 57

11 Engineering practices 59
11 Design 59
11 Correctness 60
11 Safety and cost 61
11 Maintainability 61
11 Delivery 62
11 Practices in the viewer 62
11 Known limits 62

8. System architecture

QViz is made of four parts: a Python library, the adapters that expose it, a browser viewer and the hosts that run them. This page shows what each part does and how a request travels through them. The pipeline inside the library is described in Pipeline overview, and the layout of the files in Project structure.

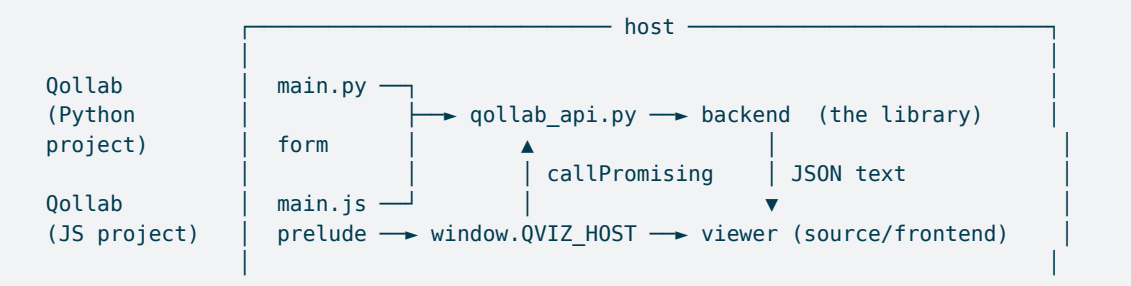
The four parts

Part	Where it lives	What it does
Library	`source/backend/`	Turns a Qiskit circuit and a backend into the JSON of the contract, and measures a step of that JSON. It has no knowledge of any host.
Adapters	`main.py`, `qollab_api.py`, `resources/scripts/server.py`	Expose the library to a host: a form in Qollab, functions called by JavaScript, or HTTP routes on a local machine. They contain no physics.
Viewer	`source/frontend/`, `source/shaders/`	Reads the JSON and draws the ions. It performs no calculation of its own; compiling and measuring always go back to Python.
Hosts	Qollab, or a local browser	Provide the browser, Pyodide and (on Qollab) the `backend` object.

Two decisions shape everything else.

1. **The contract is the only coupling.** The library produces a JSON document (see JSON contract), and the viewer consumes it. Neither side imports anything from the other.
2. **The viewer works alone.** What Qollab needs is added through optional hooks and generated code, never by rewriting the viewer (see The viewer on Qollab).

How the parts are connected



```
local          |  server.py ──► POST /api/compile, /api/sample ──► viewer     |
machine        |                                                              |
               └──────────────────────────────────────────────────────────────┘
```

Each adapter calls the same functions of `qollab_api.py`, which in turn call the four public functions of the library. For this reason, a circuit gives the same JSON on Qollab and on a local machine, as long as the backend is the same.

Three ways to run

Mode	Entry point	The viewer is	Python is reached through	Typical use
Qollab, Python project	`main.py`	absent	the form of the QPU dialog	Checking a backend and reading the JSON in the console.
Qollab, JS project	`qollab/` (built by `make qollab`)	the real viewer, bundled in `main.js`	`window.QVIZ_HOST`, which calls `qollab_api` with `callPromising`	The product: the viewer with the backend chosen by the user.
Local machine	`make server`	the real viewer, served from the repository	`POST /api/compile` and `POST /api/sample`	Developing the viewer and the library without Qollab.

The viewer can also be opened with `python -m http.server`. In that mode the Python services are missing, so it can show the files of `out/` but it cannot compile a loaded circuit or measure; it says so in its error line.

A request from end to end

The most common request is a person loading an OpenQASM file in the viewer.

1. The viewer reads the file and calls `compileQasm(text)` (`js/host.js`).
2. On Qollab, `HOST.compileQasm` calls `qollab_api.run_qasm(backend, qasm, …)` through `callPromising`. Locally, the same call is a `POST /api/compile`.
3. `run_qasm` calls `load_circuit(qasm)` (the `importer` module), which returns a `QuantumCircuit`.
4. `run_qasm` calls `compile(backend, circuit, allow_remote=False)`. The `compiler` module runs the pipeline: `provider`, `transpiler`, `translator`, `simulator`, `vectorizer`, `metrics` and `formatter`.
5. `run_qasm` calls `export(data)` and returns JSON text. A dictionary is never returned, because Pyodide would turn it into a `Map` that `JSON.stringify` cannot print.
6. The viewer parses the text, adds the circuit to its selector and draws the first step (`js/data.js`).
7. When the person presses **Medir**, the viewer calls `sampleStep`. This reaches `qollab_api.sample_step`, which calls `sample(data, step, shots)` on the same JSON. No job is sent to IonQ.

Step 4 passes `allow_remote=False` on purpose. On Qollab, an IonQ backend allows one job per run, and a viewer that loads several files would use it up with the first one.

Where each rule is enforced

Rule	Enforced by
The library never imports the injected `backend`.	The `compile` signature takes the backend as an argument; `main.py` imports `from backend import …` and never `import backend`.
Only one IonQ job per run.	`compile` validates its arguments before the transpiler runs, and `allow_remote` can disable the job.
No job is sent to paid hardware by accident.	`tomography.measure` raises unless `allow_paid=True` for a hardware backend.
Anything that Python returns to JavaScript is text.	`qollab_api` returns JSON strings, and a test checks the prelude against the functions that exist.
The viewer runs without Qollab.	Every Qollab service is read as `HOST.<name>` with a fallback.
The project fits in Qollab.	`package_qollab.py` builds a project of 52 files (the limit is 100), and a test checks the limits.

What is not connected

Some links are absent on purpose.

- **The pipeline modules do not import each other.** Only `compiler` knows all of them. See Engineering practices.
- **The viewer does not import Python code, and Python does not import the viewer.** The JSON is the only thing that crosses.
- **`main.py` is not part of the JS project.** It could compile a second time and spend a second IonQ job.

9. Project structure

This page is the map of the repository. It lists every folder, says what belongs in it, and shows which parts of the library may depend on which.

The repository

```
main.py                     Entry point of the Qollab Python project.
qollab_api.py               The functions that Qollab pages call (they return text).
requirements.txt            Dependencies for Qollab (kept equal to pyproject.toml).
pyproject.toml              Packaging, dependencies and tool settings.
Makefile                    The commands of the project (see Tooling and workflow).
.flake8                     The style rules of flake8.
.github/workflows/ci.yml    Continuous integration: lint and tests.
README.md, LICENSE          The overview of the project and its MIT license.
AGENTS.md, CLAUDE.md        The rules for code agents.
memory.md                   The hand-over between work sessions.

source/                     What is shipped: the library, the viewer and the tests.
  backend/                  The Python library.
    __init__.py             The public API: compile, export, load_circuit, sample.
    core/                   The modules (see below).
  frontend/                 The viewer.
    index.html              Markup and style only.
    js/                     The ES modules of the viewer.
  shaders/                 GLSL: the Bloch spheres, laser, lens, phonon bus and rail.
  tests/                   The pytest suite, the shared circuits and the golden file.

resources/                  What supports the project.
   examples/                     Worked circuits (Bell, GHZ, layers, the QFT adder)
and .qasm files.
  models/                   The laser model (OBJ and MTL).
   scripts/                     The demos, the generator of out/, the packager and
the server.
   make/                         Makefile.git (branch workflow) and Makefile.quarto
(this book).

documentation/              The specification, the schema and this book.
  _quarto.yml               The configuration of the book.
  docs/                     The chapters.
  diagrams/                 PlantUML sources and their SVG renders.
  schema/                   The JSON schema of the contract.
  prompts/                  spec.md and the working notes of the team.
  references/               Notes on the native gates of IonQ.

out/                        Generated: the example JSON files and their manifest.
qollab/                     Generated: the project that is uploaded to Qollab.
documentation/_book/        Generated: the rendered book.
```

The three generated folders are not committed. `make outputs`, `make qollab` and `make render` create them, and nothing in them is edited by hand.

Why `source/` and `resources/` are separate

`source/` holds what the product is made of. `resources/` holds what is needed to build, demonstrate or document it. The split has a practical effect: the setuptools configuration looks in both (`where = ["source", "resources"]`), but it installs only the packages `backend*` and `examples*`. The demos, the packager and the server are therefore never installed with the library.

The modules of the library

Everything under `source/backend/core/` follows one rule: a package per module, and a public function whose name is a verb.

```
core/
  model.py          Frozen dataclasses that travel between the stages.
  errors.py         The exception hierarchy.
  constants.py      Values that never change (for example, a full turn).
  config.py         Values that policy may change (limits, decimals, timeouts).
  to_qiskit.py      A native circuit rebuilt as a QuantumCircuit.
  provider/         detect → BackendInfo; profiles.py holds the noise profiles.
  importer/         load_circuit: OpenQASM 2 or 3 → QuantumCircuit.
  transpiler/       transpile: the circuit changed to the gates of the backend.
  translator/       translate: normalize.py and rules.py produce only native gates.
  simulator/        simulate: one file per gate matrix, and the tomography.
  vectorizer/      vectorize: spherical angles to (x, y, z) vectors and probabilities.
  metrics/          measure: fidelity, inactivity, vibration and cost.
  formatter/        format: the dictionary of the contract.
  exporter/         export: the dictionary as JSON text.
  sampler/          sample: a simulated measurement of one step.
  compiler/         compile: runs the stages in order.
```

Each package has an `__init__.py` with one line, and one module file whose name is the package name. Helpers sit next to it in the same package. For example, the translator keeps its rules in `rules.py` and its pre-processing in `normalize.py`, and the simulator keeps one matrix per file (`gpi.py`, `gpi2.py`, `ms.py`, `zz.py`, `virtualz.py`).

Module catalogue

What each module does, and where it sits in the flow. The pipeline modules are those with an arrow in the last column.

Module	Function	Role	In ⟶ out	Page
`model`	(dataclasses)	Defines every object that travels between modules.		Data model
`config`, `constants`, `errors`		Limits and policy, fixed values, and the exception hierarchy.		Configuration and errors
`importer`	`load_circuit`	Reads an OpenQASM 2 or 3 text and explains what cannot be loaded.	text ⟶ `QuantumCircuit`	Importer

Module	Function	Role	In → out	Page
`provider`	`detect`	Recognises IonQ Aria, IonQ Forte or the built-in simulator, picks the entangler and the noise profile.	backend → `BackendInfo`	Provider
`transpiler`	`transpile`	Changes the circuit to the gates the backend understands; the single IonQ dry-run job is sent here.	circuit → `TranspiledCircuit`	Transpiler
`translator`	`translate`	Rewrites every gate as GPI, GPI2, MS or ZZ, and virtual Z.	`TranspiledCircuit` → `NativeCircuit`	Translator
`simulator`	`simulate`	Computes the exact state after every native gate and reduces it to one Bloch state per qubit.	`NativeCircuit` → `SimulatedCircuit`	Simulator
`simulator.tomography`	`measure`	Estimates those states from measurements instead of computing them.	`NativeCircuit` → `SimulatedCircuit`	Tomography
`vectorizer`	`vectorize`	Spherical angles to `(x, y, z)` and the probabilities `[p0, p1]`.	`SimulatedCircuit` → `VectorizedCircuit`	Vectorizer
`metrics`	`measure`	Adds fidelity, inactivity, vibration and cost.	`VectorizedCircuit` → `MeasuredCircuit`	Metrics
`formatter`	`format`	Builds the dictionary of the JSON contract.	`MeasuredCircuit` → `dict`	Formatter and exporter
`exporter`	`export`	Writes the dictionary as JSON text, or to a file.	`dict` → `str`	Formatter and exporter
`sampler`	`sample`	Draws the result of measuring a step from the JSON; ap-	`dict` → counts	Sampler

Module	Function	Role	In → out	Page
		plies the readout error.		
`to_qiskit`	`to_qiskit`	Rebuilds a native circuit as a `QuantumCircuit`; used by the tomography, the sampler and the tests.	`NativeCircuit` → `QuantumCircuit`	Bridge to Qiskit
`compiler`	`compile`	Runs the pipeline in order and checks the arguments.	circuit and backend → `dict`	Compiler

Who may import whom

The imports of the library form a shallow tree. The table is read row by row: the module in the first column may import the ones in the second column.

Module	May import
`model`, `errors`, `constants`, `config`	Nothing from `core`. `model` uses Qiskit types.
`to_qiskit`	`model`, `constants`
`provider`	`model`
`importer`	`errors`, `config`
`transpiler`	`model`, `errors`, `config`
`translator`	`model`, `errors`, `constants`
`simulator` (and the tomography)	`model`, `errors`, `config`, `constants`, `to_qiskit`
`vectorizer`	`model`
`metrics`, `formatter`	`model`, `config`
`sampler`	`model`, `errors`, `config`, `constants`, `to_qiskit`
`exporter`	Nothing from `core`
`compiler`	Every stage, `model` and `errors`

No arrow points from one stage to another. The only module with a wide view is `compiler`. This has a direct benefit: a stage can be tested with a hand-made input, without running the stages that come before it.

The adapters sit above the library and import only its public functions.

```
main.py ──┐
          ├──► qollab_api.py ──► backend (compile, export, load_circuit, sample)
server.py ┘                                  ▲
                                             └── examples/ (the circuits shown in
the selector)
```

Where the tests are

The suite lives in `source/tests/`, with one `test_<module>.py` per module. It also covers the adapters and the tools (`test_qollab_api.py`, `test_server.py`, `test_package_qollab.py`), the examples, the performance budget and the files that keep the project consistent (`test_requirements.py` compares `requirements.txt` with `pyproject.toml`, and `test_memory.py` keeps `memory.md` short). See Tests.

10. Public API and entry points

The library exposes four functions. Three small adapters wrap them for the places where QViz runs. This page lists both, with their signatures and the reason each adapter exists.

The public API

`source/backend/__init__.py` is the only import that code outside the library needs.

```
from backend import compile, export, load_circuit, sample
```

Function	Module	Takes	Returns
`load_circuit(text)`	`importer`	OpenQASM 2 or 3 text	a `QuantumCircuit`
`compile(backend, qc, *, entangler, strategy, shots, allow_paid, allow_remote)`	`compiler`	a backend object and a circuit	the dictionary of the JSON contract
`export(data)`	`exporter`	that dictionary	JSON text
`sample(data, step, shots)`	`sampler`	the dictionary and a step	a `dict` of bit strings and counts

`__version__` is also exported. Each function is documented on the page of its module.

Two points are worth knowing before the adapters.

- `compile` receives `backend` as an argument. It never imports or creates one, because Qollab injects the object as a global of `main.py`.
- `import backend` is never written in `main.py`. It would replace the injected variable. The form `from backend import compile` is safe.

`qollab_api.py`

`qollab_api.py` holds the functions that both `main.py` and the pages of the viewer call. It exists so that the logic lives in one place. Its functions take and return plain values and **JSON text**, never dictionaries.

Function	Purpose
`CIRCUITS`	The example circuits by name: `bell`, `bell_distant`, `ghz3`, `ghz16`, `layered16`.
`build_circuit(name)`	Builds one of them; an unknown name raises `QVizError`.
`parse_entangler(value)`	Turns `"auto"`, `"ms"` or `"zz"` into `None`, `"ms"` or `"zz"`. Any other text is an error, never a silent default.

Function	Purpose
`circuits()`	The selector entries as JSON: `[{"file": …, "label": …}]`.
`run(backend, circuit, shots, entangler)`	Compiles an example and returns the JSON text.
`run_qasm(backend, qasm, shots, entangler)`	Compiles an OpenQASM text, locally (`allow_remote=False`), and returns the JSON text.
`sample_step(visualization, step, shots)`	Measures a step of a JSON text and returns `[[bitstring, count], …]` as JSON.

The reply of `sample_step` is a list of pairs and not an object. A JavaScript object reorders integer-like keys (`"10"` before `"11"`), which would lose the order from the most frequent outcome down.

main.py

`main.py` is the entry point when the project is uploaded to Qollab as a Python project.

```
def main(circuit: str = "bell", shots: int = 1024, entangler: str = "auto", step: int
= 0) -> None:
    ...
```

Because the parameters have strict annotations, Qollab draws a form for them in the QPU dialog. The function compiles an example circuit, prints the backend, the size, the time and the JSON (only its size if there are more than 30 steps), then measures one step and prints the most frequent outcomes. It is the only place in the project, apart from the scripts, that calls `print`.

The signature is on one line because Qollab requires it. For that reason, `.flake8` ignores `E501` for this file only.

resources/scripts/server.py

`make server` starts this file. It serves the repository like `python -m http.server`, and adds two routes that call the same functions as Qollab, on the built-in simulator.

Route	Body	Query	Reply
`POST /api/compile`	OpenQASM text	`shots`, `entangler`	the JSON of the visualization
`POST /api/sample`	the JSON of a visualization	`step`, `shots`	the list of pairs of `sample_step`

A circuit that cannot be compiled answers `400` with `{"error": "…"}`, and an unexpected failure answers `500` with the same shape. A negative or non-numeric `Content-Length` answers `400`, and a body over the limit answers `413`. The server listens on `127.0.0.1` only, sends `Cache-Control: no-store`, and redirects `/` to `/source/frontend/index.html`.

It is a development tool. Qollab never runs it.

The other tools

File	Purpose
`resources/scripts/package_qollab.py`	Builds `qollab/`, the JS project uploaded to Qollab. See The viewer on Qollab.
`resources/scripts/generate_outputs.py`	Writes the JSON of the example circuits to `out/`, with `out/manifest.json`.
`resources/scripts/demo_<module>.py`	A readable demonstration of one stage. See Demos.
`resources/scripts/_common.py`	The helpers that the demos share.

Choosing the right entry point

If the goal is to…	Use
Compile a circuit from Python code	`compile` and `export` from `backend`
Read a `.qasm` file	`load_circuit`, then `compile`
Measure a step of a JSON already produced	`sample`
Check a backend on Qollab and read the JSON	the Python project (`main.py`)
Show the viewer to someone	the JS project (`make qollab`), or `make server` locally

11. Engineering practices

QViz was built as a hackathon project, but it follows the habits of a maintained codebase. This page lists those habits. For each one, the rule is stated, the place where it can be seen in the code is named, and the reason is given. The mechanical style rules (imports, docstrings, line length) are on Style and conventions.

Design

One responsibility per module

Each pipeline stage does one thing and returns a new type. The translator knows nothing about simulation, and the simulator knows nothing about fidelity. A new metric is added in `metrics` and nowhere else.

Why. A reader can open one file and understand one idea. A bug is located by asking which stage produced the first wrong object.

Modules do not import each other

Only `compiler` imports the stages. The others import `model`, `errors`, `config`, `constants`, and, when a conversion is needed, `to_qiskit`. The full table is on Project structure.

Why. There are no hidden dependencies and no cycles. A stage is tested with an object made by hand, as the tests of `simulator` and `vectorizer` do.

Orchestration is separate from logic

The whole `compiler` module is 71 lines. `compile` validates the arguments, calls the stages in order and returns the result. It contains no physics. The decisions that look like logic (the unsupported provider, the tomography branch) are the choice of which stage to call, which is the job of an orchestrator.

Why. The order of the pipeline is readable in one screen. Changing it never means searching for scattered calls.

Immutable data, pure functions

Every object that crosses a stage boundary is a `@dataclass(frozen=True)` defined in `model.py`, and tuples are used instead of lists. The public functions return new objects and do not touch their input. There is no module-level state.

Why. Purity makes the functions deterministic, so they are easy to test and safe to call twice. Immutability turns an accidental write into an exception at the moment it happens, rather than a wrong number several stages later.

Make illegal states hard to express

Names such as the entangler and the gate are `Literal` types (`"ms" | "zz"`, `"gpi" | "gpi2" | "ms" | "zz" | "virtual_z"`). The functions that dispatch on a gate end with `assert_never`, in `simulator._matrix` and in `to_qiskit`.

Why. If a new gate is added to the type and a function forgets it, `assert_never` raises at once (and a type checker would flag it before that), instead of the function silently skipping the gate.

Constants and configuration are different things

`constants.py` holds values that are part of the definition (a full turn in radians). `config.py` holds values that are a policy (`MAX_QUBITS`, the number of decimals of the JSON, the timeout of the IonQ job). Each configuration value has a docstring that says why it has that number.

Why. A maintainer who wants to change a limit finds it in one place, and knows that it is safe. Nobody changes a constant by mistake.

Correctness

Test against an independent oracle

A test that compares code with a copy of itself proves little. The suite uses Qiskit as the reference instead.

What is checked	Oracle
The translation is correct	`Operator(source).equiv(Operator(to_qiskit(native)))`
The simulation is correct	`Statevector` and `partial_trace` of Qiskit
The JSON is valid	`documentation/schema/visualization.schema.json`, with `jsonschema`
The JSON has not drifted	A golden file, `source/tests/golden/bell_basic.json`

When the documentation of IonQ and its own matrices disagreed, the matrices and the tests were taken as the authority. This is recorded in `documentation/references/ionq_native_gates.md`.

Tolerances are honest

Numerical comparisons use `atol=1e-9`. A failing test is fixed at its cause; the tolerance is not loosened to make it pass. The statistical tests of the tomography use a looser tolerance, and the sampler tests pass a fixed `seed`, so a failure can be repeated.

Validate at the boundary

Data that comes from outside is checked where it enters. `compile` rejects an unknown strategy or a non-positive number of shots before it touches the transpiler, so a typo cannot spend the single IonQ job of a run. `sample` receives a JSON that a web page may have modified, so it checks every field (finite numbers, valid targets, one value per qubit) and raises a `QVizError` that names the step and the reason.

Why. Inside the library, the data is trusted. At the edge, it is not.

Fail with a precise error

Every error is an instance of `QVizError` or one of its subclasses (`UnsupportedOperationError`, `CircuitTooLargeError`, `UnsupportedProviderError`, `CircuitParseError`, `TranspilationError`). Messages say what was found and what is accepted, for example `entangler must be 'auto', 'ms' or 'zz', got 'foo'`. A bad value is never replaced silently by a default. Where the library catches a broad `Exception` to fall back, the line carries a comment that says why.

Never fail where classification is enough

`provider.detect` does not raise. A backend that is neither IonQ nor the built-in simulator is classified as `unsupported`, and `compile` decides what to do about it. The detection answers a question; the policy is somewhere else.

Safety and cost

- **One IonQ job per run.** The rule comes from Qollab. `compile` makes at most one `backend.run`, and `allow_remote=False` makes none.
- **No paid hardware by accident.** The tomography sends many circuits, so it refuses a hardware backend unless `allow_paid=True` is passed.
- **No secrets.** There are no API keys or tokens in the repository, because the published code is public.
- **Dependencies are checked.** Each one has a permissive license (MIT, BSD, Apache-2.0, ISC), and it is recorded in the specification. `requirements.txt` must equal the dependencies of `pyproject.toml`, and a test compares them.

Maintainability

Logging, not printing

Inside `source/backend/core`, messages go through `logging.getLogger(__name__)`. A caller decides whether to see them. `print` is used only in `main.py` and in the scripts.

Write the specification first

`documentation/prompts/spec.md` defines the architecture, the units, the JSON contract and a plan by module. A module is built against its section, with its tests and a demo, and it is marked *ready to validate* only when `make test` and the demo pass. When the code shows that the specification is wrong, both are corrected in the same change.

One module at a time, with a stop

The work proceeds in the order M0 to M10. After each module the author stops and reports what was done, how to validate it and what remains unverified on Qollab. The next module starts after the team agrees.

Why. A pipeline fails far from its cause. Validating each stage before building the next keeps the failures local.

Document where the knowledge is lost

Three kinds of knowledge would otherwise be forgotten, and each has a home.

Knowledge	Where it lives
Why a decision was made	The specification, and the decision notes in `documentation/prompts/`.
What is true about Qollab	`qollab.qmd` and the test table in `documentation/prompts/qollab_pruebas.md`, with the date of each measurement.
Where the project stands	`memory.md`, kept under 150 lines by a test.

Units are stated, always

An angle is radians or turns, and the unit is part of the name or the docstring. `qiskit-ionq` uses turns for the entangling angle; the conversion happens at the border, once. See Units and conventions.

Delivery

- **Continuous integration** runs the linters and the tests on every push and pull request, on Python 3.12 and 3.13.
- **Branches** follow a simple flow: `main` is stable, `development` integrates, and one `feature/<module>` branch per module. The commands are in `Makefile.git`.
- **Commits** are made when the team asks, in English, with a short first line. Hooks are never skipped and history is never rewritten on shared branches.
- **Generated folders are not edited.** `qollab/` is rebuilt by `make qollab`, and `out/` by `make outputs`.

Practices in the viewer

The same ideas apply to the JavaScript, with the limits of the platform in mind.

- **State in one place.** `js/state.js` holds all the mutable state of the viewer. No value hides in the document or in a closure.
- **Optional hooks with a fallback.** Everything Qollab adds is read as `HOST.<name>` and has a default. The viewer runs without Qollab.
- **A test per risk of the platform.** The packager has tests for each thing Qollab broke: the 100-file limit, the missing `main.py`, import cycles in the bundled modules, and the services that the prelude defines but the viewer does not read.
- **Accessibility is a feature, not an afterthought.** The *View and accessibility* panel offers profiles (focus, high contrast, colour-blind palettes, reduced motion), camera buttons for people who do not use a keyboard, and an `aria-live` announcement of each step.
- **Bilingual interface.** Texts are chosen with `t(es, en)` according to `<html lang>`.

Known limits

These limits are known and still open.

- The viewer has no automated browser test. It is checked by hand, and by headless Chromium for the bundle, and its modules are exercised with a stub DOM.
- The tomography with `qiskit-ionq` older than 1.0.3 fails on backends that are not IonQ, because the native gates carry no definition.
- The comparison of the tomography with the exact statevector is not implemented; the circuit is compiled twice.
- Only IonQ and the built-in simulator are supported. IBM and AWS backends are rejected on purpose. See the Compiler.

V

Backend foundations

12 Configuration, constants and errors . 65
12 The two constant modules . . . 65
12 The fixed constants (`constants.py`) . . . 65
12 The tunable parameters (`config.py`) . . . 65
12 The error module . . . 67

13 Data model . . . 69
13 Type aliases . . . 69
13 `NoiseProfile` . . . 70
13 `BackendInfo` . . . 70
13 `TranspiledCircuit` . . . 70
13 `NativeOp` . . . 71
13 `NativeCircuit` . . . 71
13 `QubitAngles` . . . 72
13 `SimulatedStep` and `SimulatedCircuit` . . . 72
13 `VectorizedStep` and `VectorizedCircuit` . . . 72
13 `MeasuredStep` and `MeasuredCircuit` . . . 73

14 Provider . . . 75
14 `detect(backend, entangler=None)` . . . 75
14 How the entangler is chosen . . . 75
14 How the hardware flag is set . . . 76
14 `detect_provider(backend)` . . . 76
14 The noise profiles . . . 77
14 `profile_for(provider, name, entangler)` . . . 77

12. Configuration, constants and errors

This page documents two small modules that are used by every other module, and the exception hierarchy: `source/backend/core/config.py`, `source/backend/core/constants.py` and `source/backend/core/errors.py`.

The two constant modules

The values are split into two files, because they have two different meanings.

File	Contents	May be changed?
`constants.py`	Mathematical and structural constants.	**No.** They are part of the definition of the project.
`config.py`	Policy parameters.	**Yes.** They describe a choice, not a law.

Both are imported as modules.

```
from backend.core import config, constants

limit = config.MAX_QUBITS
turn = constants.TWO_PI
```

The fixed constants (`constants.py`)

Constant	Value	Meaning
`TWO_PI`	`2 * pi`	One full turn, in radians. It is used for the turns-to-radians conversion.

These values never change. They are not configuration.

The tunable parameters (`config.py`)

Parameter	Value	Meaning
`MAX_QUBITS`	`16`	The hard cap on the circuit width.
`MAX_QASM_CHARS`	`200000`	The longest OpenQASM text that the importer reads.
`MAX_STEPS_WARNING`	`500`	If more steps than this are produced, a warning is logged.

Parameter	Value	Meaning
`VIBRATION_THRESHOLD`	`0.01`	The smallest per-qubit error that is reported as vibration.
`JSON_DECIMALS`	`6`	The number of decimals used when a floating-point number is written.
`MAX_SAMPLE_SHOTS`	`100000`	The largest number of shots that `sample` accepts in one call.
`REMOTE_JOB_TIMEOUT_S`	`60.0`	The seconds to wait for the IonQ dry-run job. When the time is over, the local transpile is used.

Which backends are accepted

`compile` accepts two kinds of backend: the IonQ machines, which are the target of the visualizer, and the local simulators, such as the built-in simulator of Qollab, which have no connectivity limit. There is no setting for this. Any other backend, such as an IBM or an AWS Braket device, is classified `"unsupported"` and rejected.

The reason is that the circuits of those providers are transpiled for a chip that is not a trapped-ion machine. The connectivity of that chip can force extra `SWAP` gates and permute the labels of the qubits, which is wrong for a visualizer of ions. The problem is described in `documentation/prompts/decision_transpilacion_no_ionq.md`.

Two points are important.

- The detection is not affected. `provider.detect` still classifies every backend, and it never fails. Only `compile` rejects the unsupported ones, right after the detection, before the transpiler can spend the single IonQ job.
- A rejected backend raises `UnsupportedProviderError`. The message is meant to be read by the user: `This machine is not currently supported (detected: default (BraketLocalBackend)). Currently supported: IonQ (Aria and Forte), the built-in simulator.` It names the backend and its class. In Qollab, the generated prelude removes the Python traceback and shows only this text.

In Qollab, this means that the IBM noisy simulators, the AWS Braket local simulator and any unrecognised backend are rejected with this error. This was checked in Qollab on 2026-10-11 with IBM Boston and AWS Braket local.

How the width limit is used

The limit is not taken from `MAX_QUBITS` alone. The backend is also asked for its own number of qubits. The effective limit is the smaller of the two.

- The cap is `MAX_QUBITS`, which is 16.
- When the backend exposes a `num_qubits` attribute with a positive integer, the effective limit is `min(MAX_QUBITS, backend.num_qubits)`.
- When the backend does not expose it, the cap of 16 is used.

In this way, a circuit is never sent to a backend that is too small for it. For example, a 6-qubit circuit is rejected by a backend that has 5 qubits.

The check is made in `transpiler._effective_max_qubits`, at the start of the pipeline.

The cap is 16 for two reasons. First, the simulator keeps the whole statevector, which has `2^n` complex numbers (a 28-qubit state alone takes 4 GiB), and every step goes through all of them; memory and time grow exponentially. Second, this is a visualizer: with more than about 16 Bloch spheres the picture is no longer readable.

The JSON schema follows the same cap: the maximum of `total_qubits` in `documentation/schema/visualization.schema.json` is 16, and a test checks that it equals `MAX_QUBITS`. If the cap is changed, the schema must be changed with it.

The error module

The file `source/backend/core/errors.py` defines a small hierarchy of exceptions. Only these exceptions are raised by the backend, so that a caller can catch all the expected failures with one `except` clause.

```
QVizError
├── UnsupportedOperationError
├── CircuitTooLargeError
├── CircuitParseError
├── UnsupportedProviderError
└── TranspilationError
```

Exception	Raised when
`QVizError`	The base class, which is used to catch any QViz error. It is also raised directly for a wrong argument or a refused action: an unknown `strategy`, `shots` that is not positive, or tomography on hardware without `allow_paid`.
`UnsupportedOperationError`	The circuit contains something that cannot be shown, for example a mid-circuit measurement, a `reset`, classical control flow, an unbound parameter, or a malformed entry in a native gate list.
`CircuitTooLargeError`	The circuit is wider than the effective limit that is described above.
`CircuitParseError`	The text of a circuit (OpenQASM) could not be read: it is empty, too long, has no valid header or has a syntax error. See the importer.
`UnsupportedProviderError`	The backend is neither an IonQ machine nor the built-in simulator. It is raised by `compile` before anything is transpiled.
`TranspilationError`	The circuit could not be transpiled for the chosen backend.

An unsupported provider is not an error of the detection

If the provider of a backend is not IonQ or the built-in simulator, `provider.detect` does not raise an error. It returns the provider `"unsupported"`. This behaviour is described on the Provider page. The backend is rejected later, by `compile`, with `UnsupportedProviderError`.

13. Data model

Every object that travels between the pipeline stages is defined in `source/backend/core/model.py`. All the objects are **frozen dataclasses**. A frozen dataclass cannot be changed after it is created, so the pipeline functions are easy to keep pure.

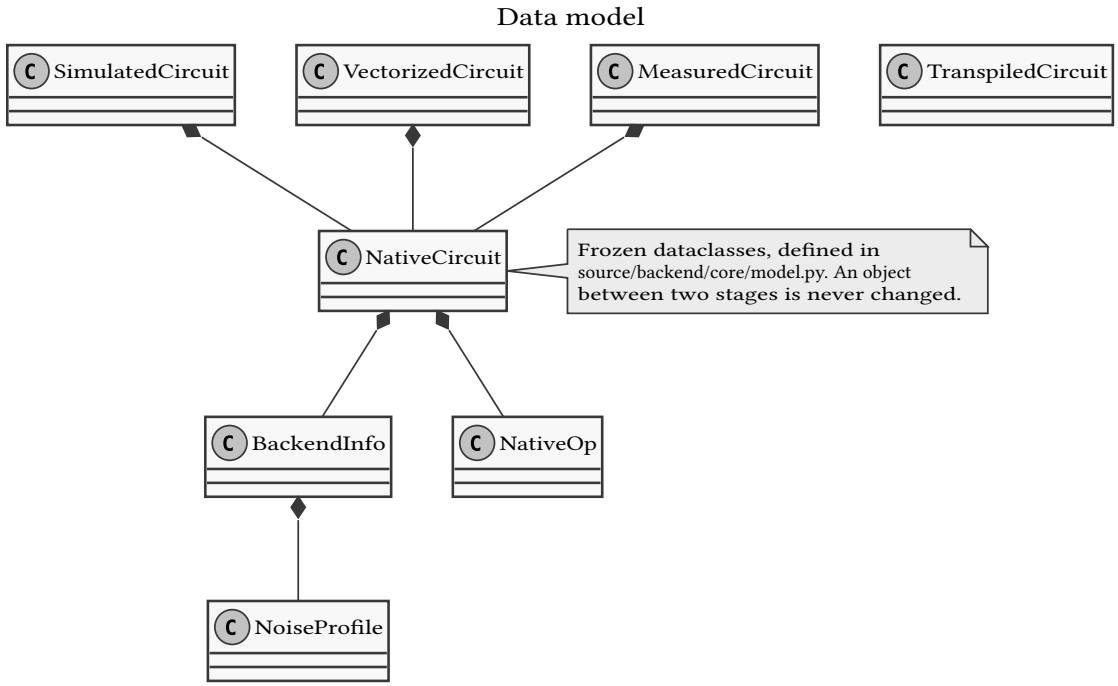


Figure 13.1: The objects of the pipeline and their relationships.

Type aliases

Alias	Allowed values	Meaning
`Provider`	`"ionq"`, `"local"`, `"unsupported"`	The family of the backend. `compile` rejects `unsupported`.
`Entangler`	`"ms"`, `"zz"`	The two-qubit native gate that is used for entanglement.
`OpName`	`"gpi"`, `"gpi2"`, `"ms"`, `"zz"`, `"virtual_z"`	The name of a native operation.
`Source`	`"ionq-native"`, `"qiskit"`	How a transpiled circuit was produced.
`Strategy`	`"statevector"`, `"tomography"`	How the state of the qubits is found.

`NoiseProfile`

The didactic noise parameters of a reference machine. All the fidelities are numbers in the range [0, 1]. The prices are in US dollars per gate-shot.

Field	Type	Meaning
`spam_error`	float	The state-preparation-and-measurement error.
`fidelity_1q`	float	The fidelity of a single-qubit gate.
`fidelity_2q`	float	The fidelity of a two-qubit gate.
`idle_error_per_step`	float	The error added to a qubit that is not used in a step.
`price_1q`	float	The price of a single-qubit gate, in USD per shot.
`price_2q`	float	The price of a two-qubit gate, in USD per shot.

`BackendInfo`

The result of the provider detection. It is produced by `provider.detect`, and it is carried through the whole pipeline.

Field	Type	Meaning
`provider`	`Provider`	The detected provider.
`name`	string	The raw name of the backend, as given by the platform.
`display_name`	string	The readable name used in the JSON.
`entangler`	`Entangler`	The target entangler, `"ms"` or `"zz"`.
`is_hardware`	bool	`True` only for real IonQ hardware, which spends credits.
`profile`	`NoiseProfile`	The noise profile of the reference machine.

`TranspiledCircuit`

The result of the transpiler. Either `circuit` or `native_ops` is set, but never both.

Field	Type	Meaning
`source`	`Source`	`"qiskit"` for a normal circuit, `"ionq-native"` for native gates.
`num_qubits`	int	The number of qubits of the transpiled circuit.
`circuit`	`QuantumCircuit` or `None`	The transpiled Qiskit circuit.
`native_ops`	tuple of dict or `None`	A raw IonQ native gate list, when the API form is received.
`layout`	tuple of int or `None`	The mapping from user qubits to physical qubits.

NativeOp

A single native operation of the intermediate representation.

Field	Type	Default	Meaning
`name`	`OpName`	required	The gate name.
`targets`	tuple of int	required	The qubits the gate acts on.
`phases`	tuple of float	`()`	Phases in radians. One value for `gpi` and `gpi2`, two for `ms`.
`angle`	float or `None`	`None`	The entanglement angle in turns, for `ms` and `zz`.
`z_rotation`	float or `None`	`None`	The rotation in radians, for `virtual_z` only.
`source_index`	int or `None`	`None`	The index of the original instruction, used for traceability.

NativeCircuit

A circuit that is made only of native operations.

Field	Type	Meaning
`backend`	`BackendInfo`	The backend information.
`num_qubits`	int	The number of qubits after compaction.
`ops`	tuple of `NativeOp`	The operations, in temporal order.

`QubitAngles`

The reduced state of one qubit, in spherical coordinates. See Units and conventions for the definitions.

Field	Type	Range	Meaning
`r`	float	$[0, 1]$	The length of the Bloch vector.
`theta`	float	$[0, \pi]$	The angle from the `+z` axis.
`phi`	float	$[0, 2\pi)$	The angle from the `+x` axis.

`SimulatedStep` and `SimulatedCircuit`

`SimulatedStep` holds the state after one operation.

Field	Type	Meaning
`index`	int	The step number, starting at 1.
`op`	`NativeOp`	The operation that was applied.
`qubits`	tuple of `QubitAngles`	The state of every qubit, after the operation.
`virtual_z`	tuple of float	The accumulated virtual Z rotation of every qubit, in radians.

`SimulatedCircuit` holds the native circuit together with all the steps.

Field	Type	Meaning
`native`	`NativeCircuit`	The simulated circuit.
`steps`	tuple of `SimulatedStep`	The steps, in order.

`VectorizedStep` and `VectorizedCircuit`

The angles are replaced by Cartesian vectors and by probabilities.

Field	Type	Meaning
`index`	int	The step number.
`op`	`NativeOp`	The operation that was applied.
`states`	tuple of `(x, y, z)`	The Bloch vector of every qubit.
`probabilities`	tuple of `(p0, p1)`	`[p(|0⟩), p(|1⟩)]` for every qubit.
`virtual_z`	tuple of float	The accumulated virtual Z rotation of every qubit.

`VectorizedCircuit` holds the native circuit together with the vectorized steps.

Field	Type	Meaning
`native`	`NativeCircuit`	The circuit.
`steps`	tuple of `VectorizedStep`	The steps, in order.

`MeasuredStep` and `MeasuredCircuit`

The vectorized step is decorated with the didactic metrics.

Field	Type	Meaning
`index`	int	The step number.
`op`	`NativeOp`	The operation that was applied.
`states`	tuple of `(x, y, z)`	The Bloch vector of every qubit.
`probabilities`	tuple of `(p0, p1)`	The probabilities for every qubit.
`virtual_z`	tuple of float	The accumulated virtual Z rotation of every qubit.
`fidelities`	tuple of float	The per-qubit fidelity, in percent.
`cumulative_fidelity`	float	The global fidelity, in percent.
`inactivity`	tuple of int	The accumulated inactivity of every qubit.
`vibration`	tuple of float	The vibration amplitude of every qubit.
`cumulative_cost`	float	The accumulated cost, in USD.

`MeasuredCircuit` holds the native circuit together with the measured steps.

Field	Type	Meaning
`native`	`NativeCircuit`	The circuit.
`steps`	tuple of `MeasuredStep`	The steps, in order.

14. Provider

The provider module is found in `source/backend/core/provider/`. It has two responsibilities. The family of the backend is recognised, and the noise profile of a reference machine is chosen. Two files are used: `provider.py` and `profiles.py`.

`detect(backend, entangler=None)`

The main function of the module. The provider, the entangler and the noise profile of a backend are returned together in one `BackendInfo` object.

Parameters

Name	Type	Default	Meaning
`backend`	any object	required	The backend that is injected by Qollab, or a local backend in a test.
`entangler`	`"ms"` or `"zz"` or `None`	`None`	The target entangler. When it is `None`, the entangler is inferred from the backend and from its name.

Returns a `BackendInfo` object. See the Data model page for its fields.

How the entangler is chosen

The following order is used. The first step that gives an answer is kept.

1. **The parameter.** When `entangler` is given, it is used as it is.
2. **The backend itself.** When the backend advertises only one native two-qubit gate, that gate is used. For example, a backend whose operation names contain `zz` but not `ms` gives `"zz"`.
3. **The machine family.** The family is read from the machine hints: `options.noise_model` first, and then the name (see below). The helper `native_2q_gate` turns a hint such as `aria-2` into `"ms"`. A hint that contains `forte` or `ideal` also gives `"zz"`.
4. **The default.** When nothing is found, `"ms"` is used.

A name that contains `ideal` is treated as a `zz` machine, because the ideal reference is paired with the fixed `zz` entangler.

The IonQ machine family

Every IonQ entry in Qollab is named `ionq_simulator`, so the name does **not** say whether the machine is an Aria (MS) or a Forte (ZZ). The family is read from `backend.options.noise_model` instead, which is `aria-2`, `forte-1`, `forte-enterprise-1` and so on. The helper `native_2q_gate` of the provider module converts that value into `ms` or `zz`: a name with `aria` gives `ms`, and a name with `forte` gives `zz`. It is our own copy of the function of `qiskit-ionq` 1.0.3 and later, because the version of `qiskit-ionq` that Qollab ships does not have it.

The machine hints are therefore collected from two places, in this order: `options.noise_model`, then the name. Each hint is tested with `native_2q_gate`, and the first hit is used. When `options.noise_model` raises an exception or holds a `NoiseModel` object, it is ignored, and the name is used.

How the hardware flag is set

`is_hardware` is `True` only for an IonQ backend that is not a simulator. A boolean attribute of the backend is preferred, because it is set by the software development kit itself. The attributes `_simulator`, `_is_simulator`, `is_simulator` and `simulator` are checked, in this order.

The name is inspected only as a last resort, when no attribute is found. This is deliberate. A name is a weak signal, and it can be misleading. The attribute is always preferred when it exists.

`detect_provider(backend)`

The family of the backend is found, and one of the three provider names is returned.

Parameter

Name	Type	Meaning
`backend`	any object	The backend to classify.

Returns one of `"ionq"`, `"local"` or `"unsupported"`.

The detection is layered. The reliable signals are checked first, and a structural fallback is used last.

The first signals

Signal	Provider
the class module starts with `qiskit_ionq`, or the name contains `ionq`	`ionq`
the module, the class or the name contains `BasicSimulator`, `basic_simulator`, `aer_simulator` or `AerSimulator`	`local`

The structural fallback

When the signals above give no answer, the operation names that the backend advertises are inspected. They are read from `target.operation_names`, from `operation_names`, or from `configuration().basis_gates`.

Operation names that are found	Provider
any of `gpi`, `gpi2`, `ms`	`ionq`
anything else (IBM, AWS Braket, an unrecognised backend)	`unsupported`

Only `gpi`, `gpi2` and `ms` are treated as IonQ-native here, because `zz` and `rxx` also exist as ordinary Qiskit gates and would produce false positives.

This fallback exists so that an IonQ backend with an unusual name, but with a clear gate set, is still classified correctly. The detection never raises an error: any other backend is classified as `"unsupported"`. It is `compile` that rejects it. See the configuration page.

The noise profiles

The file `profiles.py` contains three ready-made profiles.

Constant	Machine	Notes
`ARIA`	IonQ Aria	Used for the MS entangler.
`FORTE`	IonQ Forte	Used for the ZZ entangler.
`IDEAL`	an ideal machine	All fidelities are 1, and all prices are 0.

The numbers are simple estimates that were taken from public IonQ material. They are **not** a calibrated noise model. They can be replaced before the demo.

`profile_for(provider, name, entangler)`

The correct profile is chosen for a backend.

Parameters

Name	Type	Meaning
`provider`	string	The provider name. Only `"ionq"` uses the name of the backend.
`name`	string	The best machine hint: `options.noise_model` when it is a string, and the raw backend name otherwise. For IonQ it is used to find `aria`, `forte` or `ideal`.
`entangler`	`"ms"` or `"zz"`	The fallback entangler when the name gives no hint.

Returns a `NoiseProfile`.

The rule. For an IonQ backend, the name is checked first. A name that contains `ideal` gives the ideal profile, `forte` gives the Forte profile and `aria` gives the Aria profile. If the name gives no hint, or if the backend is not IonQ, the reference machine of the entangler is used: Aria for MS and Forte for ZZ. A non-IonQ backend is therefore shown *as if* it ran on trapped ions, whatever its name is.

VI Backend input and translation

15 **Importer** 81
15 Why it is needed 81
15 `load_circuit(text)` 81
15 Loading without spending the IonQ job 82
15 In Qollab 82

16 **Transpiler** 83
16 `transpile(info, backend, qc, *, allow_remote=True)` 83
16 What is done first 83
16 The IonQ strategies 84
16 The local simulator 84
16 The layout 85
16 The meaning of the `source` field 85

17 **Translator** 87
17 `translate(info, transpiled)` 87
17 The translation rules 87
17 The compaction of the qubits 90
17 The normalization of the angle 90
17 A note on global phases 91

18 **The `to_qiskit` bridge** 93
18 Why it is not inside the translator 93
18 `to_qiskit(native, upto=None)` 93
18 `circuit_from_ops(ops, num_qubits)` 93

15. Importer

The importer is found in `source/backend/core/importer/importer.py`. It reads a circuit written in **OpenQASM 2 or 3** and returns a Qiskit `QuantumCircuit`. It is **not** a stage of the pipeline: `compile` receives the circuit it returns, like any other circuit. It is exported from `backend` for convenience: `from backend import load_circuit`.

Why it is needed

The viewer used to list a few example circuits. A circuit that a person writes must go through the whole platform (transpilation, translation to native gates, simulation and metrics) before it reaches the JSON contract, so the viewer does not load JSON files in Qollab any more: it loads circuits. OpenQASM is the standard text format for them, and the organizers of the hackathon recommended it.

`load_circuit(text)`

Name	Type	Meaning
`text`	string	The circuit. The first statement must be `OPENQASM 2.0;` or `OPENQASM 3.0;` (comments before it are allowed).

Returns a `QuantumCircuit`.

- OpenQASM 2 is read with `qiskit.qasm2.loads`, which comes with Qiskit.
- OpenQASM 3 is read with `qiskit.qasm3.loads`, which needs the package `qiskit-qasm3-import` (MIT-compatible: Apache-2.0, with `openqasm3` and `antlr4-python3-runtime`, which are pure Python). It is imported only when a circuit in OpenQASM 3 is loaded, so OpenQASM 2 keeps working if it is missing.

Raises

Exception	When
`CircuitParseError`	The text is empty, longer than `config.MAX_QASM_CHARS` (200 000 characters), has no valid header, uses a version other than 2 or 3, or has a syntax or semantic error. The message keeps the line and column of the parser.
`UnsupportedOperationError`	The circuit has control flow (`if`, `for`, `while`, `switch`), which a step-by-step view cannot show, or an `opaque` gate, which has no definition to translate.

The limits that depend on the machine are not checked here. `compile` rejects more qubits than the limit (`CircuitTooLargeError`), mid-circuit measurements and `reset`, and parameters without a value (`input float theta;`), with `UnsupportedOperationError`. Final measurements (`c = measure q;`) are accepted and removed, as for any circuit.

Loading without spending the IonQ job

On IonQ the transpilation asks the machine for its own compilation, and that spends the single job allowed per run. A person who loads several files cannot afford it, so `qollab_api.run_qasm` calls `compile(..., allow_remote=False)`: the circuit is transpiled locally and the job is never sent. The error model (`metrics`) does not use a job: it works from the fixed profile of the machine.

In Qollab

`qollab_api.run_qasm(backend, qasm, shots, entangler)` reads the text and returns the JSON text of the visualization. The viewer calls it through `HOST.compileQasm` (see the Qollab page). The example circuits are in `resources/examples/qasm/` (`bell.qasm`, `ghz4.qasm`, `qft3.qasm` and `bell_qasm2.qasm`).

16. Transpiler

The transpiler is found in `source/backend/core/transpiler/transpiler.py`. It changes the circuit of the user into the gate set of the chosen backend. It is the first stage of the pipeline, and the only one that may call `backend.run` before the tomography stage.

`transpile(info, backend, qc, *, allow_remote=True)`

Parameters

Name	Type	Default	Meaning
`info`	`BackendInfo`	required	The backend information produced by `provider.detect`. The provider decides which strategy is used.
`backend`	any object	required	The backend object. It is passed to Qiskit, and it may be used for a dry-run job.
`qc`	`QuantumCircuit`	required	The circuit of the user. It is never changed.
`allow_remote`	bool	`True`	When it is `False`, the IonQ dry-run job is not allowed. This is used by the tomography strategy, because that strategy needs the single job for itself.

Returns a `TranspiledCircuit`.

Raises `CircuitTooLargeError` if the circuit is wider than the effective limit, `UnsupportedOperationError` if the circuit has unbound parameters (they must be bound before compiling) or a gate without a definition (for example an `opaque` gate of OpenQASM; the message names it). Both checks are made before any job can be spent, and `TranspilationError` if the circuit is empty or cannot be transpiled. The effective limit is the smaller of `MAX_QUBITS` and the number of qubits of the backend. It is described on the configuration page.

What is done first

These steps are always performed before the strategies are tried.

1. The final measurements are removed.
2. The number of qubits is checked against the effective limit, which is the smaller of `MAX_QUBITS` and the width of the backend.
3. The circuit must have at least one qubit.

4. The circuit must have no unbound parameters.
5. Every gate must have a definition, also inside the definitions of other gates. Qiskit decomposes any gate that has one into the native gates of the ion trap, so a gate without a definition (a plain `Gate` or `Instruction`, such as an `opaque` gate) is the one case that cannot be translated, and it is reported by its name.

A circuit with measurements at the end is therefore accepted. A measurement in the middle of the circuit is not removed, and it is rejected later by the translator.

The IonQ strategies

For an IonQ backend, three strategies are tried, in order. The first one that works is used.

Order	Strategy	How it works	Job spent
1	`native-local`	The circuit is transpiled locally with a native-gate backend, by `qiskit.transpile`.	none
2	`remote`	A dry-run job is sent with `backend.run(..., dry_run=True)`. The native circuit is read from `job.compiled_circuit(lang="native")`.	one
3	`local`	The standard Qiskit transpiler is used. The result is handled like a non-IonQ circuit.	none

Strategy 1 is used only when the gateset of the backend is already `"native"`. Strategy 2 is skipped when `allow_remote` is `False`. If strategy 2 fails for any reason, a warning is logged, and strategy 3 is used. The wait for the job is limited to `config.REMOTE_JOB_TIMEOUT_S` seconds, so a job that never finishes cannot block the page: when the time is over, strategy 3 is used.

The order of the strategies is important for the platform rule of one job per run. The native-local strategy spends no job. The remote strategy spends the one job that is allowed.

The local simulator

For a backend that is not IonQ, one strategy is used. `compile` only accepts the IonQ machines and the local simulators (see the configuration page), because the coupling map of a chip can permute the qubit labels. This path is therefore reached with a local simulator.

1. The circuit is transpiled with `qiskit.transpile(qc, backend=backend, optimization_level=1)`.
2. If that fails, it is transpiled again with a fixed basis and no coupling map. The basis is `["rz", "sx", "x", "cz", "cx"]`.

If both attempts fail, a `TranspilationError` is raised.

An IonQ circuit is **never** produced for a non-IonQ backend. The change between providers is a static translation, which is done by the translator.

The layout

When a layout exists, it is saved in the `layout` field. The layout maps the user qubits onto the physical qubits of the backend. It is read with `circuit.layout.initial_index_layout(filter_ancillas=True)`. The layout is used by the translator, so that unused physical qubits can be removed.

The meaning of the `source` field

The `source` field of the returned object says how the circuit was produced.

- `"ionq-native"`: the circuit already contains native gates, or native operations were returned by the IonQ API.
- `"qiskit"`: the circuit is a normal Qiskit circuit, and it must be translated.

17. Translator

The translator is found in `source/backend/core/translator/`. It changes any transpiled circuit into a sequence of native ion gates. Only `gpi`, `gpi2`, `virtual_z` and **one** entangler, either `ms` or `zz`, appear in the result. The two entanglers are never mixed.

Three files are used.

File	Responsibility
`translator.py`	The `translate` function.
`rules.py`	The translation table, one rule per gate.
`normalize.py`	The entangling angle is reduced to the range [0, 0.25] turns.

The opposite direction, rebuilding a Qiskit circuit from a native circuit, is not part of the translator. It lives in its own module, described on the `to_qiskit` page.

`translate(info, transpiled)`

A transpiled circuit is changed into a native circuit. The same translation rule is used for every backend. The only thing that changes the result is the target entangler, which is taken from `BackendInfo`.

Parameters

Name	Type	Meaning
`info`	`BackendInfo`	The backend information. Its `entangler` (`"ms"` or `"zz"`) is the two-qubit gate that is emitted.
`transpiled`	`TranspiledCircuit`	The output of the transpiler: a Qiskit circuit with its layout, or a raw IonQ native gate list.

Returns a `NativeCircuit`.

There is no class per provider. `compile` only accepts IonQ and the built-in simulator (see the configuration page), and the subclasses that existed for every provider were identical.

The translation rules

The rules are given in `rules.py`. The temporal order is the list order: the leftmost operation is applied first. All the identities are exact up to a global phase, and a global phase does not change a Bloch vector.

Source gate	Translation
`x`	`gpi(0)`
`y`	`gpi(π/2)`
`sx`, `sxdg`	`gpi2(0)`, `gpi2(π)`

Source gate	Translation
`rz(λ)`, `p(λ)`, `u1(λ)`	`virtual_z(λ)`
`z`, `s`, `sdg`, `t`, `tdg`	`virtual_z` with `π`, `π/2`, `-π/2`, `π/4`, `-π/4`
`h`	`virtual_z(π)`, `gpi2(π/2)`
`cz(a,b)`	a `zz` operation, or the equivalent MS form, followed by `virtual_z(-π/2)` on both qubits
`cx(c,t)` with the MS entangler	`gpi2(c,π/2)`, `ms(c,t)`, `gpi2(c,π)`, `gpi2(t,π)`, `gpi2(c,-π/2)`
`cx(c,t)` with the ZZ entangler	`h(t)`, `cz(c,t)`, `h(t)`
`rzz(θ)`	`zz(θ / 2π)`, then normalized
an IonQ native gate	taken directly, with the phases changed from turns to radians
`id`, `barrier`, `delay`, a final `measure`	ignored
any other one-qubit gate	decomposed with the Euler decomposition, then translated
any other two-qubit gate	decomposed onto `cz`, then translated
a gate of three or more qubits	expanded through its definition, then translated
a mid-circuit `measure`, `reset`, or classical control	`UnsupportedOperationError`

The one-qubit gates with a fixed angle (`x`, `y`, `sx`, `sxdg`, `z`, `s`, `sdg`, `t`, `tdg`) are read from three small constant tables in `rules.py`, instead of one branch each.

`emit(ops, op, qubits, entangler, source_index=None)`

One Qiskit operation is translated, and the new native operations are appended to the list `ops`.

Parameter	Type	Meaning
`ops`	list of `NativeOp`	The output list. New operations are appended to it.
`op`	Qiskit operation	The operation to translate.
`qubits`	tuple of int	The qubits the operation acts on.
`entangler`	`"ms"` or `"zz"`	The target entangler.
`source_index`	int or `None`	The index of the original instruction, copied onto each new operation.

No value is returned. The list `ops` is changed.

`emit_zz(ops, targets, angle, entangler, source_index=None)`

A `ZZ(angle)` operation is written, after the angle is normalized and after the entangler is converted when necessary.

Parameter	Type	Meaning
`ops`	list of `NativeOp`	The output list.
`targets`	tuple of int	The two qubits.
`angle`	float	The angle in turns. It is normalized inside the function.
`entangler`	`"ms"` or `"zz"`	The target entangler.
`source_index`	int or `None`	The index of the original instruction.

`emit_ms(ops, targets, phases, angle, entangler, source_index=None)`

An `MS(phases, angle)` operation is written, in the same way.

Parameter	Type	Meaning
`ops`	list of `NativeOp`	The output list.
`targets`	tuple of int	The two qubits.
`phases`	tuple of two floats	The phases in radians.
`angle`	float	The angle in turns.
`entangler`	`"ms"` or `"zz"`	The target entangler.
`source_index`	int or `None`	The index of the original instruction.

`translate_circuit(circuit, layout, entangler)`

A whole Qiskit circuit is translated, and its qubits are compacted.

Parameter	Type	Meaning
`circuit`	`QuantumCircuit`	The circuit to translate.
`layout`	tuple of int or `None`	The layout produced by the transpiler.
`entangler`	`"ms"` or `"zz"`	The target entangler.

Returns a pair `(ops, num_qubits)`, where `ops` is a list of `NativeOp` and `num_qubits` is the number of qubits after compaction.

`translate_native_ops(native_ops, num_qubits, entangler)`

A raw IonQ native gate list is translated. This form is received when the native API payload is read.

Parameter	Type	Meaning
`native_ops`	sequence of dict	Each dictionary has a `gate` key and a `target` or `targets` key. A `phase`, `phases` or `angle` key may also be present. Phases and angles are in turns.
`num_qubits`	int	The number of qubits. The targets are checked against it.
`entangler`	`"ms"` or `"zz"`	The target entangler.

Returns a list of `NativeOp`.

Raises `UnsupportedOperationError` when an entry is malformed. The list comes from a remote payload, so each entry is validated before it is used. These cases are rejected, and the message gives the index of the entry:

- an entry that is not a dictionary;
- an unknown `gate`;
- a missing, non-numeric or out-of-range target, or a repeated target;
- a number of targets that does not fit the gate (one for `gpi` and `gpi2`, two for `ms` and `zz`);
- a missing or non-numeric `phase` (`gpi`, `gpi2`), a `phases` value that does not have two numbers (`ms`), or a non-numeric `angle`.

The compaction of the qubits

A wide backend, with tens of qubits, is reduced to the qubits that are used. The rules are the following.

- The user qubits are kept, in their original order. The order is read from the layout.
- A user qubit that is never used is still kept. It is a quiet ion in `|0⟩`.
- A physical qubit that is used only for routing is appended at the end.
- A physical qubit that is not touched is removed.

One limitation is accepted. A final permutation that is caused by routing is not undone. In practice this does not happen: the local simulators and the IonQ machines have all-to-all connectivity, so the transpiler does not need routing.

The normalization of the angle

The file `normalize.py` reduces an entangling angle to the range [0, 0.25] turns. The identities that are used are exact up to a global phase.

- One turn is subtracted or added when the angle is outside the range. This is a global phase, so it is free.
- Half a turn is subtracted when the absolute value is larger than 0.25. A conjugation is added to keep the operation correct.
- A negative angle is made positive by a conjugation on the first qubit.

`normalize_zz(targets, angle)`

Parameter	Type	Meaning
`targets`	tuple of int	The two qubits.
`angle`	float	The angle in turns.

Returns a list of `NativeOp`. When the angle is zero, an empty list is returned and nothing is emitted.

`normalize_ms(targets, phases, angle)`

Parameter	Type	Meaning
`targets`	tuple of int	The two qubits.
`phases`	tuple of two floats	The phases in radians.
`angle`	float	The angle in turns.

Returns a list of `NativeOp`.

A note on global phases

Some of the identities differ from the original gate by a global phase, for example a factor of `-1` or `-i`. A global phase has no physical meaning, and it does not change any Bloch vector. The tests therefore use an equivalence check that ignores the global phase.

18. The `to_qiskit` bridge

The file `source/backend/core/to_qiskit.py` rebuilds a Qiskit circuit from a `NativeCircuit`. It is the opposite of the translator, which goes from Qiskit to native gates, and it is not a pipeline stage.

Why it is not inside the translator

Three parts of the project need this function: the tomography strategy, the sampler and the tests. The pipeline modules must not import each other, so the tomography could not import it from the `translator` package. The function is therefore placed directly in `core/`, next to `model.py`, `errors.py` and `config.py`. It only depends on the model, the constants and `qiskit-ionq`, so any module may use it.

`to_qiskit(native, upto=None)`

A Qiskit circuit is rebuilt from a `NativeCircuit`. The native gates of `qiskit-ionq` are used, and the phases are changed back from radians to turns. A `virtual_z` operation becomes an `RZ` gate.

This function is used as the oracle in the tests. It is also used by the tomography strategy, to build the prefix circuits.

Parameter	Type	Default	Meaning
`native`	`NativeCircuit`	required	The circuit to rebuild.
`upto`	int or `None`	`None`	When it is given, only the first `upto` operations are used. `upto=0` gives an empty circuit.

Returns a `QuantumCircuit`.

`circuit_from_ops(ops, num_qubits)`

The same construction, but from a plain sequence of `NativeOp` and a number of qubits, without a `NativeCircuit`. `to_qiskit` calls it. The sampler uses it directly, because it rebuilds the operations from the exported JSON and has no `NativeCircuit` (and no backend information).

Parameter	Type	Meaning
`ops`	sequence of `NativeOp`	The operations, in temporal order.
`num_qubits`	int	The width of the circuit.

VII

Backend simulation and output

19 **Simulator** 97
19 `simulate(native)` 97
19 How the state is stored 97
19 How a qubit is described 97
19 The `virtual_z` operation 97
19 The gate matrices 98

20 **Vectorizer** 99
20 `vectorize(simulated)` 99
20 The conversion 99

21 **Metrics** 101
21 `measure(vectorized, info, shots=1024)` . . 101
21 The initial fidelity 101
21 The per-step rules 101
21 What a `virtual_z` does not change 102
21 The ideal profile 102
21 The guarantees 102

22 **Formatter and exporter** 103
22 `format(measured)` 103
22 `export(data, path=None)` 103
22 Why the dictionary and the string are separated 104

19. Simulator

The simulator is found in `source/backend/core/simulator/`. It computes the state of every qubit after every native operation. The statevector is computed exactly; no noise is added at this stage. Each native gate has its own file, and a `matrix` function is defined in each one.

`simulate(native)`

The main function of the module.

Parameters

Name	Type	Default	Meaning
`native`	`NativeCircuit`	required	The circuit to simulate.

Returns a `SimulatedCircuit`.

This function always computes the exact statevector. The tomography method is documented on its own page; the `compiler` chooses between the two before this function is called.

How the state is stored

The state is a NumPy array of complex numbers with the shape `(2,) * n`, where `n` is the number of qubits. It is initialised to `|0...0)`. The Qiskit little-endian order is used: qubit `q` lives on the axis `n - 1 - q`.

A gate is applied with a tensor contraction over the axes of its target qubits. The matrix of a two-qubit gate is written in the convention of the reference document, where the first target is the most significant qubit.

How a qubit is described

After each operation, the reduced density matrix of the qubits that the operation acted on is found by tracing out the other qubits. The other qubits keep their previous description, because a unitary on some qubits cannot change the reduced state of the rest. This makes each step cost the work of one or two qubits instead of all of them. The Bloch components are then read from it.

$$x = 2\,\mathrm{Re}(\rho_{01}), \qquad y = -2\,\mathrm{Im}(\rho_{01}), \qquad z = \rho_{00} - \rho_{11} \tag{19.1}$$

The length `r` and the two angles `theta` and `phi` are computed from `(x, y, z)`. The angle `phi` is wrapped to the range $[0, 2\pi)$. When `r` is close to zero, both angles are set to zero. When `theta` is close to 0 or to π, `phi` is set to zero.

The `virtual_z` operation

A `virtual_z` operation is applied to the state as a real `RZ` rotation, so that the vector turns around the Z axis. This is the visible click of a virtual Z gate. The accumulated rotation of each qubit is stored, and it is wrapped to the range $[0, 2\pi)$.

The gate matrices

Each file defines one function. The phases are in radians. The angle of `ms` and `zz` is in turns.

`gpi.matrix(phase)`

```
[[0, exp(-i*phase)],
 [exp(i*phase), 0]]
```

Parameter: `phase` is the rotation phase, in radians. **Returns** a 2x2 NumPy array.
`gpi(phase)` is a rotation of π around the axis `(cos phase, sin phase, 0)`.

`gpi2.matrix(phase)`

```
1/sqrt(2) * [[1, -i*exp(-i*phase)],
             [-i*exp(i*phase), 1]]
```

Parameter: `phase` is the rotation phase, in radians. **Returns** a 2x2 NumPy array.
`gpi2(phase)` is a rotation of π/2 around the same axis family. For example, `gpi2(pi/2)` is equal to `RY(pi/2)`.

`ms.matrix(phase0, phase1, angle)`

```
cos(pi*angle)   on the diagonal,
-i*exp(-i*(phase0 + phase1))*sin(pi*angle)   at |00><11| and |11><00|,
-i*exp(-i*(phase0 - phase1))*sin(pi*angle)   at |01><10| and |10><01|.
```

Parameter	Unit	Meaning
`phase0`	radians	The phase of the first target qubit.
`phase1`	radians	The phase of the second target qubit.
`angle`	turns	The entanglement angle, in [0, 0.25].

Returns a 4x4 NumPy array.

`zz.matrix(angle)`

```
diag(exp(-i*pi*angle), exp(i*pi*angle),
     exp(i*pi*angle), exp(-i*pi*angle))
```

Parameter: `angle` is the entanglement angle, in turns. **Returns** a 4x4 NumPy array.

`virtualz.matrix(rotation)`

```
diag(exp(-i*rotation/2), exp(i*rotation/2))
```

Parameter: `rotation` is the rotation angle, in radians. **Returns** a 2x2 NumPy array.

20. Vectorizer

The vectorizer is found in `source/backend/core/vectorizer/vectorizer.py`. It changes the spherical coordinates of the simulator into Cartesian coordinates, and it computes the two probabilities of each qubit.

`vectorize(simulated)`

Parameter

Name	Type	Meaning
`simulated`	`SimulatedCircuit`	The output of the simulator.

Returns a `VectorizedCircuit`.

The conversion

For every qubit, the spherical angles are changed into a Cartesian vector.

$$x = r \sin\theta \cos\phi, \qquad y = r \sin\theta \sin\phi, \qquad z = r \cos\theta \tag{20.1}$$

The length `r` is not always 1. For a single qubit in a pure state, `r` is exactly 1. When the qubit is entangled with another qubit, the reduced state is mixed, and `r` is smaller than 1. The value `r` is therefore clipped to 1 before the conversion, only as a guard against a tiny rounding error that could make it slightly larger than 1. The two probabilities are then computed from `z`.

$$p(|0\rangle) = \frac{1+z}{2}, \qquad p(|1\rangle) = \frac{1-z}{2} \tag{20.2}$$

This conversion is the exact inverse of the one that is used by the simulator. The reason for passing through the spherical form is that the user interface also shows the angles `theta` and `phi`.

21. Metrics

The metrics module is found in `source/backend/core/metrics/metrics.py`. It adds the didactic figures to each step: the fidelities, the inactivity, the vibration and the estimated cost. This is a simple documented model, and it is **not** a simulation of noise.

`measure(vectorized, info, shots=1024)`

Parameters

Name	Type	Default	Meaning
`vectorized`	`VectorizedCircuit`	required	The output of the vectorizer.
`info`	`BackendInfo`	required	The backend information. The noise profile is read from it.
`shots`	int	`1024`	The number of shots. It is used only for the cost.

Returns a `MeasuredCircuit`.

The initial fidelity

The global fidelity starts at the initial value.

$$\text{initial_fidelity} = 100 \times (1 - \text{spam_error}) \tag{21.1}$$

The per-step rules

For each step, one fidelity factor `f` is chosen for the operation.

Operation	`f`	Price
`virtual_z`	1	0
`gpi`, `gpi2`	`fidelity_1q`	`price_1q`
`ms`, `zz`	`fidelity_2q`	`price_2q`

The following updates are then made.

- For every target qubit, the per-qubit fidelity is multiplied by `f`.
- For every other qubit, when the step is not a `virtual_z`, the per-qubit fidelity is multiplied by `1 - idle_error_per_step`, and the inactivity of the qubit is increased by one.
- The global fidelity is multiplied by `f`, and then by `1 - idle_error_per_step` once for each inactive qubit.
- The accumulated cost is increased by `shots * price`.
- The vibration of a qubit is `1 - fidelity / 100` when that error is at least `VIBRATION_THRESHOLD`, and zero otherwise.

The fidelities that are reported are percentages (0 to 100), as in the JSON. The noise profiles store fractions (0 to 1), so the module converts between the two.

What a `virtual_z` does not change

A `virtual_z` operation lasts no time. It therefore changes neither the fidelities nor the inactivity. Its fidelity factor is 1, and its price is 0. The accumulated values are left unchanged.

The ideal profile

When the ideal profile is used, every fidelity is 1 and every price is 0. All the fidelities stay at 100 percent, the cost stays at 0, and the vibration stays at 0.

The guarantees

- The accumulated fidelity never increases.
- The accumulated cost never decreases.
- The inactivity never decreases.

22. Formatter and exporter

Two modules close the pipeline. The formatter builds the JSON-ready dictionary, and the exporter writes it as a JSON string.

`format(measured)`

The function is found in `source/backend/core/formatter/formatter.py`. Because `format` is also the name of a Python built-in, the function is usually imported as a module: `from backend.core.formatter import formatter`, and called as `formatter.format(...)`.

Parameter

Name	Type	Meaning
`measured`	`MeasuredCircuit`	The output of the metrics stage.

Returns a `dict` that follows the JSON contract exactly.

What the function does

1. The four top-level keys are written: `backend`, `total_qubits`, `initial_fidelity` and `steps`.
2. One entry is written for each step.
3. A field that does not apply is left out. For example, `phases` is written only for `gpi`, `gpi2` and `ms`, and `angle` is written only for `ms` and `zz`.
4. Every floating-point number is rounded to `JSON_DECIMALS` decimals.
5. The value `-0.0` is written as `0.0`.

The full list of keys is given in the JSON contract page.

`export(data, path=None)`

The function is found in `source/backend/core/exporter/exporter.py`.

Parameters

Name	Type	Default	Meaning
`data`	`dict`	required	The dictionary that is returned by `format`.
`path`	string or `None`	`None`	When it is given, the JSON is also written to this file.

Returns the JSON as a string.

The dictionary is written with two-space indentation, so that a person can read it easily. A **simple array** is kept on one line: a list of numbers, strings, booleans or nulls, or a list of such lists. For example, `states` is written as `[[0.0, 0.0, 1.0], [0.0, 0.0, 1.0]]`, and `targets` as `[0]`. Objects, and arrays that contain objects, stay indented. The non-ASCII characters are kept as they are, so that an arrow → is not escaped.

Raises `ValueError` if the dictionary contains `NaN` or an infinity. They are not valid JSON, and `JSON.parse` would reject the whole string, so it is better to fail here.

Why the dictionary and the string are separated

The `compile` function returns a dictionary, not a string. A dictionary is easy to test. The `export` function then produces the string.

From JavaScript, the string is the natural form. The string is parsed with `JSON.parse`, and the known problem of a Python dictionary becoming a JavaScript `Map` in Pyodide is avoided.

VIII Backend orchestration and extras

23 **Compiler** 107
23 `compile(backend, qc, *, entangler=None, strategy="statevector", shots=1024, allow_paid=False, allow_remote=True)` . . . 107
23 The steps that are performed 108
23 A complete example 108

24 **Sampler** 111
24 Why it is needed 111
24 `sample(data, step, shots=1024, *, seed=None)` 111
24 How the counts are computed 112
24 `probabilities(data, step)` 112
24 The Qollab functions 112
24 The measurement of the viewer 112

25 **Tomography** 115
25 Why the state is measured, and not computed . 115
25 `measure(native, backend, info, *, shots=1024, allow_paid=False)` 115
25 Why the circuit is cut into prefixes 115
25 Why three bases are needed 116
25 Why all the circuits are placed in one call 116
25 How an expectation is found 116
25 Known limitations and conflicts 116

23. Compiler

The compiler is found in `source/backend/core/compiler/compiler.py`. It is the only module that knows all the other modules. It connects the stages of the pipeline, and it returns the final dictionary.

`compile(backend, qc, *, entangler=None, strategy="statevector", shots=1024, allow_paid=False, allow_remote=True)`

This is the main entry point of the library. It is exported from `backend` for convenience: `from backend import compile`.

Parameters

Name	Type	Default	Meaning
`backend`	any object	required	The backend that is injected by Qollab, or a local backend in a test.
`qc`	`QuantumCircuit`	required	The circuit of the user. It is never changed.
`entangler`	`"ms"`, `"zz"` or `None`	`None`	The target entangler. When it is `None`, it is inferred from the backend.
`strategy`	string	`"statevector"`	`"statevector"` computes the state exactly. `"tomography"` measures the state on the backend.
`shots`	int	`1024`	The number of shots. It must be positive. It is used for the cost, and for the tomography.
`allow_paid`	bool	`False`	It must be set to `True` before tomography is allowed on a backend that is classified as hardware, because such a backend may spend credits.
`allow_remote`	bool	`True`	When it is `False`, the IonQ dry-run job is never sent and the circuit is transpiled lo-

Name	Type	Default	Meaning
			cally. Circuits loaded from OpenQASM use it, so any number of files can be loaded in one run.

Returns a `dict` that follows the JSON contract.

Raises

Exception	When
`CircuitTooLargeError`	The circuit is wider than the effective limit (see the configuration page).
`UnsupportedOperationError`	The circuit contains an operation that cannot be shown.
`TranspilationError`	The circuit cannot be transpiled.
`UnsupportedProviderError`	The backend is neither an IonQ machine nor the built-in simulator. The message says that the machine is not currently supported, names it, and lists the supported ones.
`QVizError`	An unknown strategy is requested, `shots` is not positive, or tomography is requested on hardware without `allow_paid`.

The steps that are performed

The same order is always used. Before the first step, `strategy` and `shots` are validated, so that a wrong argument never spends the single IonQ job.

1. The provider is detected: `provider.detect(backend, entangler)`. If the provider is `"unsupported"`, `UnsupportedProviderError` is raised here, before anything is transpiled and before any job is spent.
2. The circuit is transpiled: `transpiler.transpile(...)`.
3. The circuit is translated: `translator.translate(info, transpiled)`.
4. The state is found.
   - For `"statevector"`, the simulator is used.
   - For `"tomography"`, the tomography stage is used.
5. The state is vectorized: `vectorizer.vectorize(...)`.
6. The metrics are added: `metrics.measure(...)`.
7. The dictionary is formatted: `formatter.format(...)`.

The remote transpile is disabled when the tomography strategy is used, because the tomography stage needs the single allowed job for itself, and when the caller passes `allow_remote=False`. Both are done with the same switch.

A complete example

```
from qiskit import QuantumCircuit
from backend import compile

circuit = QuantumCircuit(2)
```

```
circuit.h(0)
circuit.cx(0, 1)

data = compile(backend, circuit, shots=1024)
first_step = data["steps"][0]
```

24. Sampler

The sampler is found in `source/backend/core/sampler/sampler.py`. It simulates a measurement of all the qubits at any step of a visualization, and it returns the counts of each bitstring. It is **not** a stage of the pipeline: it works on the dictionary that `compile` returns, and it is exported from `backend` for convenience: `from backend import sample`.

Why it is needed

The `states` and `probabilities` fields of the JSON are **marginal**: they describe each qubit alone. The marginals do not determine the joint distribution. For a Bell pair, both qubits have `p(|1⟩) = 0.5`, but only the outcomes `00` and `11` can be measured. Drawing every qubit independently from its marginal gives `00`, `01`, `10` and `11` with 25 % each, which is wrong.

The joint distribution is the diagonal of the statevector, `|⟨z|ψ⟩|²`. It does not need a new field in the JSON, because the exported steps already contain every native operation with its parameters. The sampler rebuilds the circuit from them, and simulates it. A check on six circuits (Bell, GHZ of 16 qubits, an adder and random circuits of up to 16 qubits) gave a difference of at most `1e-6`, which is the rounding of the JSON.

`sample(data, step, shots=1024, *, seed=None)`

Parameters

Name	Type	Default	Meaning
`data`	`dict`	required	The dictionary returned by `compile`, or `json.loads` of the exported text. It is never changed.
`step`	int	required	The number of operations that are applied before the measurement. `0` is the initial state, and `len(data["steps"])` is the final state.
`shots`	int	`1024`	The number of measurements, from 1 to `config.MAX_SAMPLE_SHO` (100 000).
`seed`	int or `None`	`None`	A seed for the random numbers. The same seed gives the same counts.

Returns a `dict[str, int]` with the counts of each bitstring, from the most to the least frequent. The bitstring is little-endian: qubit 0 is the rightmost character, as in Qiskit.

Raises `QVizError` when `step` or `shots` is not valid, and when the data is malformed (an unknown operation, a target out of range, a missing field, a number that is not finite, or more qubits than `MAX_QUBITS`). The message of a malformed step gives its index.

How the counts are computed

1. **The circuit is rebuilt.** Each step gives a native operation: `gpi` and `gpi2` with their phase, `ms` with two phases and the angle, and `zz` with the angle. The rotation of a `virtual_z` is the difference between its accumulated `virtual_z` value and the previous one (a full turn only changes the global phase).
2. **The statevector is computed** with Qiskit, through the `to_qiskit` bridge. The ideal probabilities are `|ψ|²`. The index of the vector is the value of the bitstring.
3. **The ideal outcomes are drawn** from those probabilities. The correlations of the entanglement are therefore kept.
4. **The noise is added.** Each qubit is flipped independently with the probability of its read-out error.

The noise model

The model is didactic, like the metrics. The fidelity `f` of a qubit at the chosen step (the `fidelities` field, in percent) is a read-out error of `(1 - f) / 2`. This is the same as pulling the probability of the qubit towards 0.5 by the factor `f`: `p' = 0.5 + (p - 0.5) · f`. A noisier qubit gives a flatter distribution.

The state-preparation-and-measurement error, `1 - initial_fidelity`, is added as a second, independent flip. This is why a freshly prepared Bell pair is not measured without errors. For two independent flips with probabilities `a` and `b`, the probability that the bit is flipped is `a + b - 2ab`.

At step 0, only the SPAM error applies, because the fidelities start at 100 %.

`probabilities(data, step)`

The ideal joint distribution, before the noise. It returns a NumPy array of `2**n` entries, indexed by the value of the bitstring, with qubit 0 as the least significant bit. It is used by the tests, and it can be used to draw the exact distribution.

The Qollab functions

The file `qollab_api.py`, in the project root, has the two functions that a JS page calls with `callPromising`. They return **text**, because Pyodide turns a `dict` into a JS `Map` that `JSON.stringify` cannot print.

Function	Meaning
`run(backend, circuit="bell", shots=1024, entangler="auto")`	Compiles an example circuit (`bell`, `ghz3`, `ghz16` or `layered16`) and returns the JSON text.
`sample_step(visualization, step, shots=1024)`	Measures a step of that text. The reply is a JSON list of `[bitstring, count]` pairs, from the most to the least frequent.

The reply is a list of pairs and not an object on purpose. JavaScript sorts the integer-like keys of an object (`"10"` and `"11"` come before `"00"`), so the order of the counts would be lost.

No job is sent when a step is measured, so the rule of one job per run is not affected on the IonQ backends.

The measurement of the viewer

The `Medir` button of the viewer always uses `sample`: on Qollab, where a click can call Python (checked on 2026-10-10), and locally through `make server`, which runs the same function. The viewer has no

measurement of its own, because drawing every qubit independently from the marginals does not model entanglement. See the viewer on Qollab.

25. Tomography

The tomography stage is found in `source/backend/core/simulator/tomography.py`. It is an alternative to the exact simulator. Instead of computing the state, the three Pauli expectations of each qubit are **measured** on the backend. The effect of the real noise is therefore seen.

The tomography is selected with `strategy="tomography"` in `compile`.

Why the state is measured, and not computed

The exact simulator gives the ideal state. It cannot show the error of the real device. The tomography is used when the effect of the noise is of interest.

A run with `strategy="tomography"` only measures: it does not compute the ideal state as well, and the output has no comparison between the two. To see the error of the device, the same circuit is compiled twice, once with each strategy, and the two outputs are compared.

`measure(native, backend, info, *, shots=1024, allow_paid=False)`

Parameters

Name	Type	Default	Meaning
`native`	`NativeCircuit`	required	The circuit to measure.
`backend`	any object	required	The backend that is used for the measurement.
`info`	`BackendInfo`	required	The backend information. The hardware flag is checked.
`shots`	int	`1024`	The number of shots for each circuit.
`allow_paid`	bool	`False`	It must be `True` before a paid backend is used.

Returns a `SimulatedCircuit`, so that the rest of the pipeline is unchanged.

Raises `QVizError` when the backend is classified as hardware and `allow_paid` is `False`. The guard is deliberately conservative: it is not possible to know from the code whether the chosen backend spends credits, so the safe default is to ask for confirmation. When the backend is free, the simple fix is to pass `allow_paid=True`.

Why the circuit is cut into prefixes

The state after every gate must be shown, not only the final state. A backend can only be measured, so it cannot be asked for the state after gate `k` of a long circuit.

The solution is to build a **prefix circuit** for every `k`. A prefix contains the first `k` operations, and it is measured. In this way, the state after each operation is obtained, one measurement at a time.

A prefix is built with `to_qiskit(native, upto=k)` (see the `to_qiskit` page). The first operations are the same as in the full circuit, so the measured state is the state that the full circuit would have at that point.

Why three bases are needed

A single measurement in the Z basis gives only one number, the expectation `⟨Z⟩`. The other two components of the Bloch vector are not visible in that basis.

A rotation before the measurement is used to bring another axis into the Z basis. Three cases are needed.

Basis	Preparation before the measurement	What is measured
Z	nothing	`⟨Z⟩`
X	a Hadamard gate on every qubit	`⟨X⟩`
Y	an `Sdg` gate, then a Hadamard, on every qubit	`⟨Y⟩`

The reason for the rotations is the following. A Hadamard gate maps the `x` axis onto the `z` axis, because `H Z H = X`. An `Sdg` followed by a Hadamard maps the `y` axis onto the `z` axis. The three expectations `(x, y, z)` fully describe the reduced state of one qubit.

Why all the circuits are placed in one call

The number of circuits is `3 * N`, where `N` is the number of native operations. On IonQ, only **one** job is allowed per run. All the circuits are therefore placed in a single call to `backend.run(list_of_circuits, shots=shots)`. A second call would stop the run with the message "only one job is allowed".

For a backend that is not IonQ, the circuits are transpiled for that backend before they are sent. No job limit applies to the local simulators, but the same single call is used everywhere, so that the code is the same for every backend.

A warning is logged, because the run time grows with `shots * 3 * N` on a noisy simulator.

How an expectation is found

The counts of a measurement are used. For one qubit `q`, the bit of the qubit is read from each bitstring. The rightmost character is qubit 0, as everywhere in the project.

$$\langle \sigma \rangle_q = \frac{n_0 - n_1}{\text{shots}} \tag{25.1}$$

where `n_0` and `n_1` are the numbers of shots in which the bit of qubit `q` is 0 or 1. The counts are marginalised, so the other qubits do not matter.

The three expectations give `x`, `y` and `z`. The vector is clipped to the unit ball, and the angles are computed in the same way as in the exact simulator, so that the output has the same shape.

Known limitations and conflicts

- On IonQ, the remote strategy of the transpiler would also spend the single job. The transpiler is therefore called with `allow_remote=False` when the tomography is used, and the local transpile strategy is taken.
- On a backend that is not IonQ, the native gates have to be transpiled by Qiskit, which needs `qiskit-ionq` 1.0.3 or newer (the older versions give the gates no definition). With an older version a `TranspilationError` that says so is raised. On IonQ the circuits are sent as they are, and the version does not matter.
- The tomography is statistical. The error on each component is about `1 / sqrt(shots)`. A larger number of shots gives a better estimate.

- The number of circuits grows quickly. For a circuit with many gates, the run time and the cost grow with `3 * N`.
- On a backend that spends credits, `allow_paid=True` is required. This is a deliberate guard, and it may be relaxed when the backend is known to be free.

IX Examples

26 **The QFT adder** 121
26 The quantum Fourier transform 121
26 The Draper adder 121
26 The smallest adder 122
26 Running the example 122
26 A note on the number of steps 122

27 **Performance circuits** 123
27 `ghz(count)` 123
27 `layered(count, depth=4, seed=1234)` 123
27 Generating the files 123
27 Opening the files 123

26. The QFT adder

The worked examples are found in the `resources/examples/` directory. They are plain Qiskit circuits that can be compiled by the backend and shown in the viewer. The most interesting one is the **QFT adder**: two numbers are added with the quantum Fourier transform.

The quantum Fourier transform

The quantum Fourier transform (QFT) changes the computational basis into the Fourier basis. On a basis state `|a⟩` of `n` qubits, it is defined as follows.

$$\mathrm{QFT}|a\rangle = \frac{1}{\sqrt{2^n}} \sum_{b=0}^{2^n-1} e^{2\pi i\, ab/2^n}\, |b\rangle \tag{26.1}$$

The circuit is built from three simple pieces.

1. A Hadamard gate on each qubit.
2. A controlled phase rotation for each pair of qubits.
3. A set of swaps that reverses the order of the qubits at the end.

The controlled phase between the qubit `high` and the qubit `low` has the angle `pi / 2**(high - low)`. The inverse transform is the same circuit in reverse, with the sign of every phase changed.

The functions are `apply_qft`, `apply_inverse_qft`, `qft` and `inverse_qft` in `resources/examples/qft.py`. The `qft(count)` and `inverse_qft(count)` functions return a standalone circuit for a given number of qubits.

The Draper adder

Addition can be hidden inside the Fourier transform. The reason is the following property of the transform: a shift of the input multiplies the output by a phase.

$$\mathrm{QFT}|a+b\rangle = \mathrm{QFT}|a\rangle \cdot e^{2\pi i\, b\,(\cdot)/2^n} \tag{26.2}$$

In the Fourier basis, the addition of `b` is therefore only a **phase**. The phases are added with controlled phase rotations, and the inverse transform is applied at the end. No carry logic is needed.

The circuit of `qft_adder(count)` acts on `2 * count` qubits.

Register	Qubits	Role
`a`	the first `count` qubits	The number that is added to. It becomes `(a + b) mod 2**count`.
`b`	the last `count` qubits	The number that is added. It is left unchanged.

The phase for the pair `(a_j, b_k)` is `2 * pi / 2**(count - k - j)`. This angle is exactly the phase that the bit `b_k` gives to the Fourier component of `a_j`. The exponent `count - k - j` combines the weight `2**k` of `b_k` with the weight `2**j` of `a_j` inside the transform of `2**count` states.

The circuit has three parts, in this order.

```
1. QFT on the a register
2. controlled phase b_k -> a_j  for every pair
3. inverse QFT on the a register
```

The smallest adder

With `count = 1`, the circuit uses exactly **two qubits**, one number each, and the sum is taken modulo 2. This is the literal "adder of two qubits". The controlled phase in this case is a full `CZ`-like rotation, and the circuit is equivalent to a single `CNOT`, which is the expected result for one-bit addition modulo 2.

With `count = 2`, two two-bit numbers are added, and the circuit uses four qubits. The result is `(a + b) mod 4`.

Running the example

```
make demo M=adder
```

The script prints the Fourier matrix for two qubits, then compiles the adder for three input pairs. A JSON file is written for each pair, for example `out/adder_n2_1_1.json`. The file can be opened with the viewer.

The result is also checked by the test suite.

```
.venv/bin/python -m pytest source/tests/test_examples.py
```

The test prepares every input pair, runs the adder, and confirms that the first register holds `(a + b) mod 2**count` in every case.

A note on the number of steps

The adder is not optimised. The `count = 2` case becomes about 190 native operations after the translation, because every controlled phase is expanded into the native gate set. The `count = 1` case is much smaller. This is expected and it is useful for the visualiser: each step can be watched one at a time.

27. Performance circuits

Two wide circuits are provided to test the performance. They are found in `resources/examples/performance.py`. They are used to check that many Bloch spheres can be drawn at the same time, and that a long JSON file is handled without a problem.

`ghz(count)`

A GHZ state is prepared on `count` qubits. The circuit is shallow: one Hadamard gate, and a chain of CNOT gates. Every qubit is fully entangled with the others, so every Bloch vector has length zero. This is a good first test for the number of spheres, because the state is easy to verify.

`layered(count, depth=4, seed=1234)`

A random layered circuit is built. Each layer is a rotation on every qubit, followed by a brickwork of CZ gates. The rotations give non-trivial Bloch vectors, and the depth controls the number of native steps. A seed is used, so the circuit is reproducible.

Generating the files

```
make outputs
```

The script compiles a small set of circuits and writes the JSON files to `out/`. It also writes `out/manifest.json`, which lists every file with a readable label. The viewer reads that manifest, so a new file appears in the selector as soon as it is generated.

File	Qubits	Native steps
`bell.json`	2	7
`ghz3.json`	3	12
`ghz16.json`	16	77
`layered16.json`	16	676
`adder_n2_1_1.json`	4	187

The numbers are given for the built-in simulator. The 16-qubit circuits are the performance tests. The `layered16` circuit is the heaviest one: it has 16 spheres and 676 steps.

Opening the files

```
make outputs
python -m http.server 8000
# then open http://localhost:8000/source/frontend/index.html
```

The file `source/frontend/index.html` is connected to the generated files. It loads `out/manifest.json`, fills the selector with the available files, and opens the default one. The other files can be chosen in the

selector. The number of qubits and the number of steps are shown, together with a live frame-rate counter, so that the performance can be watched while the view is rotated.

X

Frontend

28 Viewer architecture 127
28 Layers 127
28 Shared state 127
28 From a file to the screen 128
28 Coordinates 129
28 The shaders 129
28 Layers of the scene 129
28 The host 130
28 Testing without a browser 130

29 Basic viewer 131
29 The two-qubit gate 132
29 The surface-trap chip 133
29 The horizontal layer 133
29 Preparing the ions (the intro) 133
29 The laboratory environment 133
29 The interface 134
29 What is shown 134
29 How a file is loaded 135
29 The controls 135
29 The View and accessibility panel 136
29 The sound of the gates 137
29 The circuit timeline 138
29 When a circuit cannot be shown 138
29 The measurement 139
29 How the animation works 139
29 The fields that are read 139
29 The files of the viewer 139

30 The viewer on Qollab 143
30 At a glance 143
30 The rule 143
30 What Qollab does and does not allow 143
30 How it is built 144
30 The prelude of `main.js` 145
30 The hooks of the viewer 145
30 The measurement 146
30 `qollab_api.py` 146
30 Problems found on Qollab and how they were solved 146
30 How it was verified 147
30 Rules for future changes 147

28. Viewer architecture

The viewer is plain JavaScript with ES modules and three.js. There is no build step and no framework. This page explains how the modules are layered, how a step reaches the screen, and what each shader draws. The list of files is on Basic viewer, and the integration with the platform is on The viewer on Qollab.

Layers

The modules of `source/frontend/js` fall into five layers. Imports mostly point downwards, from the entry to the foundation.

```
entry          main.js
                 |
interface      controls.js   panels.js   timeline.js   viewpanel.js   camerapad.js
                 |             |            |              |              |
data           data.js ──────── host.js ──────────────────  settings.js
                 |
scene          scene.js ── laser.js  phonon.js  trap.js  horizontal.js
                 |          measurement.js  selection.js  environment.js
                 |
foundation    state.js  config.js  utils.js  shaders.js  three-addons.js  demo.js
```

Layer	Modules	Responsibility
Foundation	`state`, `config`, `utils`, `shaders`, `three-addons`, `demo`	Constants, shared state, small helpers and loading. They import almost nothing.
Scene	`scene`, `laser`, `phonon`, `trap`, `horizontal`, `measurement`, `selection`, `environment`	Build the three.js objects and update them every frame.
Data	`data`, `host`, `settings`	Load a visualization, talk to Python, and keep the view options.
Interface	`controls`, `panels`, `timeline`, `viewpanel`, `camerapad`	The buttons, panels and the timeline strip.
Entry	`main`	Wires everything at start-up.

Some imports go sideways or upwards, for example `data.js` imports `panels.js` and `timeline.js` to refresh them when the step changes, and `scene.js` imports `settings.js` for the palettes. The one rule that is kept strictly is that **there are no import cycles**. The bundler of the Qollab package depends on this, because it puts the modules in the order of their imports, and a test fails if a cycle appears (for the timeline and for the view panel).

Shared state

`state.js` exports one object, `state`. Every module reads and writes it, and no state hides anywhere else. It is grouped by what it describes.

Field	Holds
`view`	The view options (palette, what is shown, light, motion), set by `settings.js`.
`data`, `stepIndex`	The loaded visualization and the current position in it.
`playing`, `playTimer`, `pulseMs`	The automatic play and the length of the current laser pulse.
`qubits`	One entry per ion: the animated vector and its target.
`gate`	The two-qubit gate that is running, its two ions and its time window.
`measure`, `intro`	The measurement animation and the start-up pulse.
`hovered`	The ions of the gate under the pointer in the timeline.
`shaders`	The shader sources, loaded once.
`three`	The three.js objects: renderer, scene, camera and the attributes that are updated each frame.

From a file to the screen

```
JSON text ─► loadData(obj) ─► buildScene(n) ─► updateStep(0)
                  │                                  │
                  │                       sets targets, laser pulse, gate,
                  │                       entangled pair, panels, timeline
                  ▼
            state.data                    animate(time)  ◄── requestAnimationFrame
                                             │
                                 moves each vector toward its target,
                                 writes the instance attributes,
                                 updates the layers, renders
```

1. **`loadData`** (in `data.js`) stores the visualization, builds the scene for the number of qubits, builds the timeline and starts the intro.
2. **`updateStep`** (in `data.js`) runs when the step changes. It does not draw. It sets the *targets*: where each Bloch vector must end, which ions are the targets of the gate, the colour and length of the laser pulse (`laserPulseFor`), the window of a two-qubit gate and the pair that is entangled. It also refreshes the panels.
3. **`animate`** (in `scene.js`) runs on every frame. It moves each vector 18 % of the way to its target, writes the result into the instanced attribute of the spheres, writes the intensity of each laser beam, calls the update function of every layer, and renders.

Separating the two means a step is a change of state, and the animation is a function of time. Playing, stepping back, jumping on the timeline and loading a file all use the same `updateStep`.

Coordinates

The backend uses the standard Bloch convention, with `|0⟩` at `+z`. three.js has `y` up. The conversion happens in one place, in `animate`: the Bloch vector `(x, y, z)` is written to the shader as `(x, z, -y)`, so `|0⟩` points up. The chain of ions runs along `x` at `y = 0`.

The shaders

The shaders are written in GLSL in `source/shaders/`, one pair per effect. They are loaded by `shaders.js` (over HTTP locally, or from the host on Qollab, where they are embedded in `main.js`).

Pair	Mesh	What it draws
`bloch.vert` / `bloch.frag`	One `InstancedMesh`, one instance per ion	The Bloch sphere: shell, great circles, axes, the state vector and its tip, the halo of an entangled ion and the red tint of decoherence. Per-instance attributes: `instancePsi`, `instanceEntangled`, `instanceDecoherence`.
`laser.vert` / `laser.frag`	One instanced box per ion	A volumetric laser beam, found by ray marching inside the box. `instanceActive` is the intensity of each beam; it is `0` unless the gate of the step fires on that ion.
`lens.vert` / `lens.frag`	The lens of each laser head	Glass with a dark base and chromatic edges.
`bus.vert` / `bus.frag`	A tube between the two targets of an MS or ZZ gate	The phonon bus: two waves of different frequency that travel along the tube.
`rail.vert` / `rail.frag`	A thin thread along the chain	The guide rail: a faint blue line that glows amber between the two ions of a running two-qubit gate.

The options of the *View and accessibility* panel reach the sphere shader as uniforms (`uAxisX`, `uVecColor`, `uSphereGain`, `uVectorWidth`, `uDetail`, `uMotion`, `uHalo`, among others). A test fails if the shader declares a uniform that the scene does not set.

Reserved words matter in GLSL. A variable called `active` stopped the guide rail from compiling once, and the shaders use `intensity` instead.

Layers of the scene

Each file of the *scene* layer builds one group of objects and exposes a `create…` function, called once, and an `update…` function, called on every frame.

Module	Builds
`laser.js`	The laser heads that hang from a rail, the volumetric beams, and the colour and length of the pulse of each gate.

Module	Builds
`trap.js`	The chip of the surface trap: substrate, gold pads, field rings; lights the electrodes of the target ions.
`phonon.js`	The phonon bus, the jiggle of the ions and the synchronized halo after a two-qubit gate.
`horizontal.js`	The global beam, the guide rail and the side optics, and the intro pulse.
`measurement.js`	The fluorescence animation: readout beam, flares of the ions in `|1⟩`, photons and state badges.
`selection.js`	Four arcs around the ions of the gate under the pointer in the timeline.
`environment.js`	The optical table, the coils, the viewport frame, the fog and the lights.

Laser pulses depend on the gate: GPI2 lasts one base pulse (650 ms), GPI two, and MS or ZZ three, with the entangling pair in neon orange. A virtual Z fires no laser, because it is a change of the reference frame and not a physical operation. The automatic play waits for the pulse to end, so a long entangling gate is not cut.

The host

`config.js` reads `window.QVIZ_HOST` once, as `HOST`. `host.js` is the only module that talks to Python.

```
compileQasm(text) -> HOST.compileQasm  or  POST /api/compile
sampleStep(data, step, shots) -> HOST.sample  or  POST /api/sample
```

With no `HOST` and no server, both functions fail with a message that the error line shows. The full table of hooks is on The viewer on Qollab.

Testing without a browser

The test suite has no browser. `test_package_qollab.py` checks the viewer through what is generated from it.

- The package is built and inspected: the file count, the absence of `main.py`, the license, the libraries, and that every service of the prelude is read by the viewer.
- The generated `main.js` and each module are checked for syntax with `node --check`. These tests are skipped when `node` is not installed.
- Small pieces of JavaScript are run with `node` on their own, for example the view options (their profiles and their storage, with a stub of `localStorage`) and the function that turns a Python traceback into a readable message.
- Source checks keep the layers honest: the timeline may import only `config`, `state` and `utils`, and every uniform that the sphere shader declares must be set by the scene.

Visual changes are checked by eye, locally and on Qollab, and with headless Chromium when a rendering problem has to be reproduced.

29. Basic viewer

A basic viewer is included for the JSON file. It is a tool to see the output of the backend, and it is not the final frontend. It is written in plain JavaScript: `source/frontend/index.html` draws 3D Bloch spheres with shaders, and it loads three.js from a CDN.

The viewer is connected to the JSON files that the project generates. The files are listed in `out/manifest.json`, which is written by `make outputs`. The viewer reads the manifest, fills a selector with the available files, and opens the default one. A new file appears in the selector as soon as it is generated.

One three-dimensional Bloch sphere is drawn for every qubit, and the vector of each sphere is animated from one step to the next. The ions are placed in a single **chain**, one after another, so that the view resembles an ion trap; a grid is not used. The scene can be rotated by dragging, and the zoom is changed with the mouse wheel. The camera can also be moved with the keyboard: `W`, `A`, `S` and `D` move it on the horizontal plane (relative to where it looks), the space bar flies up and the left control goes down. The number of qubits and the number of steps are shown, together with a live frame-rate counter, so that the performance can be watched.

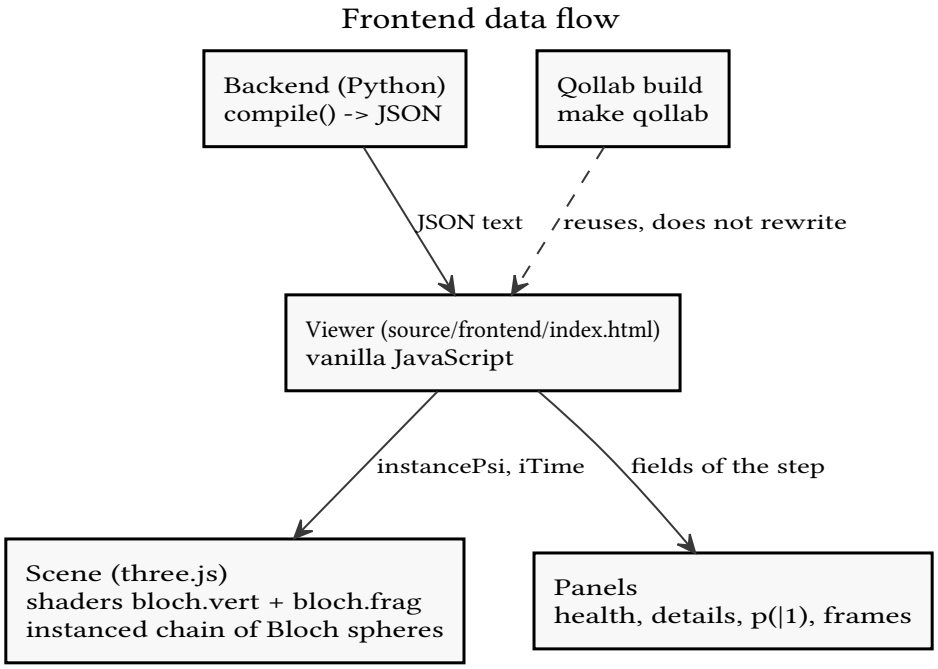


Figure 29.1: How the frontend is connected to the backend.

The spheres are drawn with the **shaders in the `source/shaders/` folder** (`bloch.vert` and `bloch.frag`). The fragment shader is a volumetric raymarcher: the axes, the state vector, the angle sweeps, the sphere shell and the dotted grid are all computed from the ray. The vertex shader is instanced, so one draw call covers every qubit. Each instance carries its own state vector in the `instancePsi` attribute, and the animation is driven by the `iTime` uniform. The `cameraPosition` uniform, which three.js provides automatically for a `ShaderMaterial`, is used to trace the rays.

The two **angle sweeps** follow the state vector: a wedge in the equator that spans from the `+x` axis to the azimuth of the vector, and a wedge in the vertical half-plane of that azimuth that spans the polar angle. Both are drawn in the same plane and with the same sign as the vector, so they always point at it.

Because the shaders are separate files, and because the JSON files are read over HTTP, the 3D viewer must be served by a local server. The commands are given below.

The **laser beams** are drawn as well. Every ion has a lens above it, and a volumetric beam connects the lens to the ion. The beam is a second instanced mesh, with its own shaders (`laser.vert` and `laser.frag`): a box that bounds the segment is raymarched, and the glow is computed from the distance to the beam axis. One box is placed per ion, with the same instance matrix as its sphere, so the beam is centred on its own ion. The colour follows the species of the backend: magenta for ytterbium (Aria, Harmony), green for barium (Forte), and violet when the species is unknown. The beam is cut by the two uniforms `uLaserMinY` and `uLaserMaxY`, which are set from `source/frontend/js/config.js` (`LASER_MIN_Y` and `LASER_MAX_Y`), so that it meets the lens exactly. `LASER_MAX_Y` is `LENS_Y`, so it follows the height of the device.

The lens is a detailed **laser head**: a fiber connector, a mounting flange with bolts, a body with a focusing ring, a front barrel and a glass aperture, built by `createLaserHead` in `source/frontend/js/laser.js`. The heads are not floating: they hang from a **rail** above the chain, which is held by a post at each end.

A beam is **dark by default**. It is fired only for the target ions of a native gate, and only while the step is applied, so no beam is left on. Each instance carries its own intensity in the `instanceActive` attribute, and moving to a step that does not fire the laser cancels a pulse that is still fading.

The pulse depends on the gate. The two-qubit entanglers fire both targets for three times the length of a GPI2, with a colour of their own (neon orange), so a two-qubit gate is told apart from a single-qubit one.

Gate	Ions	Colour	Duration
GPI2	target	species colour	`LASER_PULSE_MS` (650 ms)
GPI	target	species colour	`LASER_GPI_FACTOR` × `LASER_PULSE_MS` (2 ×, 1300 ms)
MS, ZZ	both targets	`LASER_ENTANGLER_COLOR` (neon orange)	`LASER_ENTANGLER_FACTOR` × `LASER_PULSE_MS` (3 ×, 1950 ms)
VirtualZ	none	—	—

The species colour is magenta for ytterbium (Aria, Harmony), green for barium (Forte) and violet when the species is unknown; it is returned by `laserColorFor` in `source/frontend/js/laser.js`. The length and the colour of every pulse are returned by `laserPulseFor` in the same file, from the constants in `source/frontend/js/config.js`. The pulse has a flat top: it rises and falls in the first and last 15 % of its length.

The automatic play waits for the pulse to finish before it moves to the next step, so a long MS or ZZ pulse is never cut in the middle. The wait is `PLAY_INTERVAL_MS` (700 ms) between steps, or the length of the pulse plus `PLAY_MARGIN_MS` (150 ms) when the gate fires one.

The two-qubit gate

A two-qubit gate (MS or ZZ) is shown with three elements, because the two ions do not exchange light: the whole chain is one elastic spring, and the coupling travels as a vibration (a phonon).

- **The phonon bus.** While the two lasers are on, an energy tube joins the two targets (`source/shaders/bus.vert` and `bus.frag`, built by `source/frontend/js/phonon.js`). The tube is drawn between the centres of the two ions, so it **passes through** the ions in between: the coupling is not local, it uses the whole chain. Its colour is the entangler colour, and two waves of different frequency run along it, so the flow reads as energy going back and forth.
- **The jiggle.** While the pulse runs, the ions between the two targets (and the targets themselves) swing up and down, the targets more than the ions in between, which resonate with the chain. The chain settles when the pulse ends.
- **The synchronized halo.** Once the pulse ends, every entangled ion carries a halo in the entangler colour that pulses in step (they share the time, so the pulses are exactly in sync). An ion is entangled when its Bloch vector is shorter than `ENTANGLED_R` (see below), so the halo follows the real state and not the last two-qubit gate that ran: after a SWAP the halo is gone.

The halo is drawn by the sphere shader (`source/shaders/bloch.frag`), which reads the `instanceEntangled` attribute of each instance.

The example circuit `bell_distant.json` (a Bell pair on q0 and q2) shows this directly: the entangler acts on the two ends, and the bus and the jiggle reach the ion in between, even though it is not a target.

The surface-trap chip

The ions are held by a microfabricated **surface-trap chip** below the chain (`source/frontend/js/trap.js`):

- A dark, mirror-like **silicon substrate** gives contrast to the lasers and the gold. It rests on four legs down to the optical table, so it is not floating.
- Two rows of **gold electrode pads**, segmented along the chain, shape the trapping potential on both sides of a central channel. The electrodes under an ion glow cyan while that ion is a target of the current step.
- A glowing **field ring** (a torus) encircles every ion: it is the radio-frequency quadrupole that holds it. The rings tilt slowly, and the ring of a target ion glows brighter.

The chain runs along X at `y = 0`, so the top of the substrate sits below it and the ions float above the chip.

The horizontal layer

At the height of the chain there is a second layer of optics (`source/frontend/js/horizontal.js`):

- A **global Raman beam**, a cylinder that runs along the chain and passes through every ion, from one side optic to the other. It is invisible at rest and flashes with every gate (single- or two-qubit), in step with the vertical lasers. It is also emitted once at the start of the simulation, to show the ions being prepared (the intro, see below).
- A thin **phonon guide rail** threads the ion centres: it is the confinement axis, and the vibrational continuum. It is a faint blue thread at rest; between the two ions of a two-qubit gate it glows amber and pulses, with wave rings travelling along the segment.
- The **side optics** close the layer: a readout objective lens at one end of the chain and a mirror mount at the other. They sit far enough from the end ions, and each rests on a stand down to the optical table. The pink glass of the readout lens sits flush on its barrel.

Preparing the ions (the intro)

When a visualization loads, the viewer shows how the global beam is emitted and the ions are prepared: for a second and a half the global beam pulses (the cylinder that crosses the chain) while every ion glows once, as if being initialised to $|0\rangle$. The pulse lasts `INTRO_MS` (1500 ms, `config.js`), it is a visual cue and it does not touch the data: the simulation still starts at step 0.

The laboratory environment

The scene is set inside an ultra-high-vacuum chamber on an optical table (`source/frontend/js/environment.js`):

- A dark anodised **optical breadboard** with a grid of threaded holes sits below everything.
- Two thick **copper Helmholtz coils** (above and below the trap) give the static magnetic field that defines the quantisation axis; steel posts hold them to the table.
- Every vertical laser is born from a detailed **laser head** above the ion (see the laser paragraph above).
- A polished **viewport ring** frames the scene, so the viewer feels inside the chamber. It has a floor anchor at each of its eight bends, like the supports of an above-ground pool: a post down to the table and a collar that embraces the ring.
- **Fog** (`FogExp2`) and a blue ambient light give the chamber its depth.

The interface

The interface uses a small, dark palette: **Ink** (`#121016`) for the background, **Plum** (`#211824`) for the panels and the bars, **Muted Violet** (`#48334f`) for the thin borders, **Neon Rose** (`#f05291`) for the primary actions and the accents, **Dusty Lilac** (`#b8a4c5`) for the secondary text, and **Warm White** (`#f1ebef`) for the main text. The rose is used sparingly, so the important elements stand out. The type is Inter, and the panels have thin borders, moderate radii and short transitions.

What is shown

- One Bloch sphere per qubit. The vector of the sphere is the `(x, y, z)` vector of that qubit at the current step. Its length is the length `r` of the reduced state: it is shorter than one when the qubit is entangled, and it is absent when the qubit is fully entangled (`r` is zero). Since the register is in a pure state, a qubit is entangled with the rest exactly when `r < 1`, so no other data is needed. An entangled qubit has no state of its own: the arrow is the *reduced* state (the average that a measurement of that qubit alone would give), which is mixed and lies inside the ball. It is drawn dimmer the shorter it is, and its tip and its two angle sweeps, which mark one definite pure state, fade out with it (the `purity` of `bloch.frag`). In a Bell pair `r` is zero and nothing is drawn; with partial entanglement (for example `r = 0.71` in the QFT) a faint arrow remains.
- **The vector moves like the gate.** A single-qubit gate is a rotation, so the arrow turns along the surface of the sphere and keeps its length: GPI is a half turn and GPI2 a quarter turn about the equatorial axis `(cos φ, sin φ, 0)`, and the VirtualZ turns about `z` (`js/motion.js`). The turn lasts as long as the laser pulse. A straight line between the two ends would cross the ball and make a pure state look entangled half way. The two-qubit gates do change the length, so there it is blended while the direction swings along the shortest arc. The axis and the angle are checked against the end state (843 single-qubit steps of the example circuits match), and stepping back turns the opposite way.
- The axes follow the legend: x red, z blue and vertical (|0⟩ up), y green.
- The target qubits of the step are highlighted in yellow, and the corresponding sphere is brighter.
- Each ion carries a **decoherence heat**: its colour shifts towards red as it spends the circuit idle. The heat is `1 − exp(−idle / DECOHERENCE_IDLE_STEPS)`, where `DECOHERENCE_IDLE_STEPS` is 30 and `idle` is the `inactivity` of the qubit (the steps in which no gate addressed it). The count is absolute and not a fraction of the circuit: coherence decays with the idle time itself (`exp(−t / T₂)`), so the same idle count gives the same colour in a short and in a long circuit. The scale is exaggerated on purpose, since the real idle error of the profiles (10^{-4} per step) would not be visible. The ions nobody uses are the ones that turn red, and the colour only grows. No backend change is needed.
- A volumetric **laser beam** above each ion, from the lens down to the ion. It is dark by default and fires only for the target ions of a native gate: the species colour for GPI and GPI2, and a neon-orange colour for MS and ZZ, which lasts three times longer (see the laser paragraph above).
- The probability `p(|1⟩)` of each qubit — the probability of measuring **1**, not 0 — is shown with a bar in the **Probabilities** panel.
- A **Simulation details** panel on the left shows the model (with the ion species it references, when it is a simulator), the initial fidelity, the number of qubits and the number of steps, and the accumulated fidelity of the current step: the word for it (*Fidelidad* or *Fidelity*, from the language of the page), the percentage, a bar and the **total error** (the fidelity that was lost, `100 − fidelity`). The bar is green above 99 %, amber above 95 % and red below, and the error value takes the same colour. Below it, the **accumulated cost** of the current step is shown.
- When a gate is applied, the fidelity that it subtracted (the difference with the previous step, for example `−0.53`) is popped over the panel as a red number that floats up and fades, like a damage number in a game. At step 0 the panel shows the initial fidelity of the backend.
- The **Step details** panel shows a large **operation badge** (colour-coded by gate family) with a one-line description, the target qubits, and the fields of the step with readable names (*VirtualZ*, *Fidelidad acumulada*, *Costo acumulado*, …). The per-qubit fields (VirtualZ, inactivity, fidelity and vibration) are shown as a table with one row per qubit, so every value is read at a glance.

The **Simulation details** panel is on the left; the **Probabilities** and **Step details** panels are on the right. Each one is collapsible: its title is a button, and clicking it shows or hides the body. The titles follow the language of the page (*Detalles de la simulación* / *Simulation details*, *Probabilidades* / *Probabilities* and *Detalle del paso* / *Step details*).

The +z axis is drawn upwards, and |0⟩ is at the top. This follows the standard convention that is used by the backend.

The expressions in the panels (the qubit labels, the kets, p(|1⟩) and the readout correction) are written in the LaTeX style as **inline MathML**, so the browser typesets them natively, with no maths library and nothing fetched at run time. The helpers qubitMath, ketMath, probOneMath and probCorrectionMath in source/frontend/js/utils.js build them. On the 3D canvas, where MathML cannot be drawn, the same expressions use the Unicode subscripts and angle brackets (q_0, |0⟩) from qubitLabel and ket.

How a file is loaded

The viewer does not load JSON files from the user: a circuit must go through the platform (transpilation, translation to native gates, simulation and metrics) before it can be shown. There are three ways to see a circuit.

1. **The selector.** It lists the examples. Served over HTTP, they are the files of ../out/manifest.json; in Qollab, they are compiled by the backend. If nothing is found, a small example that is stored inside the page is used.
2. **The load button** (*Cargar circuito*). A circuit written in OpenQASM 2 or 3 (.qasm) is chosen, compiled by the backend and added to the selector.
3. **Drag and drop.** The same file is dropped anywhere on the page.

Loading a circuit needs Python to compile it. In Qollab, the page calls the platform (HOST.compileQasm, see the Qollab page). Locally, run the server of the project, which compiles with the same code on the built-in simulator (see below). With a plain python -m http.server the viewer shows the examples, and loading a circuit says that the server cannot compile.

A JSON file can be produced with the generator, which also writes the manifest.

```
make outputs
```

The files are written to out/, together with out/manifest.json. Any of them can then be chosen in the selector of source/frontend/index.html, or dragged onto a viewer.

The pages are served by a small local server, so that the shaders and the JSON files can be read.

```
make outputs
make server            # or: make server PORT=9000
make open              # like server, but also opens the browser
# then open http://localhost:8000/source/frontend/index.html
```

make server runs resources/scripts/server.py. It serves the repository like python -m http.server, and it adds POST /api/compile: the body is the OpenQASM text, the optional query has shots and entangler (auto, ms or zz), and the reply is the JSON of the visualization. It calls qollab_api.run_qasm, so the result is what Qollab gives, except that the machine is always the built-in simulator. A circuit that cannot be compiled answers 400 with {"error": "..."}, which the viewer shows in its error line. The server listens on 127.0.0.1 only, and it asks the browser not to cache.

The controls

The controls that run the simulation (the step buttons, the play button, the reset, the slider and the step counter) are in a bar at the **bottom** of the page; the file selector, the load button and the measurement are at the top.

Control	Effect
Load circuit	Opens the file selector for an OpenQASM file (.qasm).
Example	Loads the small example that is embedded in the page.

Control	Effect
Left arrow	Moves to the previous step.
Play / Pause	Plays the steps automatically, or pauses. It reads *Reanudar* when paused in the middle and *Reiniciar* once the last step is reached.
Right arrow	Moves to the next step.
Reset	Stops the play and returns to step 0.
Slider	Jumps directly to a step. Step 0 is the initial state.
Step counter	Shows the current step and the total (`paso 3 / 24`).
Medir	Measures the current step and shows the distribution of the results.
shots	The number of shots used by Medir (1024 by default). The − and + buttons beside it change it by one.
Drag (left button)	Rotates the camera around the chain.
Drag (right button)	Moves the camera in the plane of the screen; the menu of the browser does not open.
Wheel	Zooms in and out.
`W`, `A`, `S`, `D`	Moves the camera on the horizontal plane.
Space	Flies the camera up.
Left control	Moves the camera down.
Camera buttons	Rotate, tilt, zoom and centre the camera (hold to keep moving); see *The View panel*.
View and accessibility	Opens the panel with the profiles and the view options.

Step 0 is the initial state, in which every qubit is in `|0⟩`. It is not part of the JSON; it is added by the viewer, in the same way as the frontend is expected to do.

The View and accessibility panel

The **View and accessibility** button is always at the right of the top bar, with the universal-access icon. It opens a panel with the options that make the scene easier to read. A profile is a quick set of values; below them, every option can be changed by itself (the profile then reads as custom). **Restablecer** goes back to the default profile. Esc, a click outside or the button close the panel.

Profile	What it does
Predeterminado	The default look: a solid sphere with a dim shell and a thick, bright vector.

Profile	What it does
Enfoque en el vector	Hides the environment, the trap, the horizontal layer, the lasers and the phonon bus, dims the light and thickens the vector.
Alto contraste	A palette of saturated colours on a dark scene without the environment, and stronger text and borders in the interface.
Daltonismo	The Okabe and Ito palette (orange, sky blue and yellow axes, a pink vector) and the numbers of each vector.
Movimiento reducido	No jiggle, halo, wobble of the sphere, sweep of the moving light or animated transitions. It is the starting value when the system asks for less motion (`prefers-reduced-motion`).

The options are in five sections:

- **Color.** The palette of the axes, the vector and the sphere: default, Okabe and Ito, high contrast and monochrome. The legend of the axes follows it.
- **Esfera y vector.** The blending of the sphere (*solid* covers what is behind it; *glow* adds to it, which washes out to white over a bright background), the level of detail (full, simple without dots or sweeps, outline only), the glow of the sphere, the brightness and the thickness of the vector, whether the spheres are drawn in front of everything, and whether the `(x, y, z)` of each vector is written under its sphere.
- **Qué se muestra.** Each group of the scene can be hidden: the environment, the trap chip, the horizontal layer, the vertical lasers, the phonon bus, the jiggle, the halo, the qubit labels, the legend and the camera buttons.
- **Luz y movimiento.** The brightness of the scene (the exposure), the intensity of the lights, the brightness of the vertical lasers and of the global beam (to see them dimmer without hiding them, which *Qué se muestra* does) and the reduced motion.
- **Interfaz.** The size of the text (90 to 130 %, which also widens the side panels) and a stronger text and border contrast.

The choice is kept in the browser (`localStorage`, key `qviz.view`); without storage the viewer starts with the defaults and works the same. `settings.js` holds the values and imports no module of the scene; `scene.js` applies them (`applyViewToScene`: the uniforms of the sphere shader, the visibility of the groups, the lights and the exposure) and `viewpanel.js` draws the panel.

The camera with the mouse alone. Everything that the keyboard does can be done with the pointer: the left button drags to rotate, the right button drags to move, and the wheel zooms. The pad at the bottom right of the scene repeats it with buttons, in four rows: rotate, tilt, move (left, right, up, down) and zoom with the centre button. A button keeps acting while it is held, with the mouse, the touch screen or the keyboard (Enter or Space on a focused button), so the camera can be moved without the letter keys and without a right button. The step is announced to screen readers in a polite live region (*Paso 3 de 7: GPI2 en q0. Fidelidad 99.44 %.*).

The sound of the gates

Every native gate has a sound, synthesised in the browser with the Web Audio API (`js/sound.js`, no audio files, so it also works in the bundle of Qollab). The length of the tone follows the laser pulse, so the sound ends when the beam does.

Gate	Sound
GPI2	A thin, high laser zap: a bright tone that falls fast in pitch over a faint, buzzing beam (the shortest pulse).

Gate	Sound
GPI	The same zap, lower in pitch and twice as long as the GPI2.
MS and ZZ	A long swell of two detuned tones that beat, with a slow vibrato and a rising filter; it lasts as long as the pulse (three times the GPI2).
VirtualZ	A click. It is a software phase, so there is no laser and no tone.

Only a step forward by one makes a sound (Play, the next button, the arrow keys and a click on the next step of the timeline); dragging the slider or jumping does not, and a new step cuts the previous sound with a short fade. Browsers only allow audio after a gesture, so the first sound comes with the first click; if the browser has no audio, the viewer works without it. The **Sound** section of the View and accessibility panel turns the sound off and sets the volume (both are kept in the browser).

The circuit timeline

A strip between the scene and the playback bar shows the **transpiled circuit**: one wire per qubit and one column per step, with the native gates in order (GPI, GPI2, MS, ZZ and VirtualZ as a dashed `Z`), in the palette of the interface: plum boxes for the single-qubit gates, neon rose for the entanglers (like the badge of the step details) and a muted dashed outline for VirtualZ. A two-qubit gate is a box on each target joined by a vertical line, which crosses the wires in between when the qubits are not adjacent. It reads only `steps[].operation`, `targets` and `step`, so the JSON contract is not changed.

- The gates that were applied are filled, the current one has a warm white border and a soft glow, and the ones to come are dimmed. A lilac column marks the current step.
- A click on a gate goes to its step and stops the play. The strip scrolls by itself to keep the current step in the middle. A tooltip gives the step, the operation, the qubits and the accumulated fidelity.
- While the pointer is over a gate (or a box has the keyboard focus), a **selection marker** fades in around its ions: four thin neon rose arcs that face the camera and turn slowly, like the selection of a unit in a game. A two-qubit gate marks both ions. It is `selection.js`, driven by `state.hovered`.
- For **long circuits**, the **–** and **+** buttons in the header change the zoom (labelled boxes, small boxes, thin marks with a step number every 10), and the choice is remembered. The background of the strip can be dragged to pan it, the wheel moves it sideways when there is nothing to scroll vertically, and a click on a step number goes to that step. The strip scrolls by itself, so a play over hundreds of steps keeps the current one in the middle.
- The keys left and right change the step, and Home and End go to the first and the last one (unless the focus is in a field).
- It is open at the start. The **Hide** button (with a chevron) at the left of its header, next to the playback buttons, closes it, and the choice is remembered in `localStorage` when the page allows it. The scene resizes with a `ResizeObserver`. Below 860 px wide it is hidden.

When a circuit cannot be shown

The viewer always opens, even when the compilation fails. A failure puts a short red line under the controls and a **card over the scene** (`reportError` in `utils.js`) that cannot be missed; the cross closes it, and a successful load removes it. A machine that QViz does not support (IBM, AWS or any other that is not IonQ or the built-in simulator) gets its own card: “Máquina no soportada”, what to do (choose IonQ Aria or Forte, or the built-in simulator) and, in small text, the message of the backend with the machine that was detected. The Qollab prelude tells the two apart because it keeps the class of the Python error in `error.kind` (`UnsupportedProviderError`); any other error shows the generic card “No se pudo mostrar el circuito” with its message. The same card is used when a file that was loaded or chosen from the list fails.

The measurement

The `Medir` button draws a number of shots from the probabilities of the current step, and shows the distribution of the resulting bitstrings. The measurement is done in the browser, from the fields of the JSON, so no job is sent to a backend.

The measurement is also shown in the scene as **state-dependent fluorescence** (`source/frontend/js/measurement.js`), the way a trapped-ion measurement really works. The individual vertical lasers are not fired; the horizontal layer does the illumination.

1. A broad **readout beam** floods the whole chain (the global beam turns green and bright).
2. An ion in `|1⟩` is on resonance and scatters photons, so it **glows**; an ion in `|0⟩` is off resonance and **stays dark**.
3. The photons are collected by the **side readout lens**: a stream of particles flies from every `|1⟩` ion into the lens.
4. The outcome is resolved: a **`|0⟩` or `|1⟩` badge** appears above every ion.

The animation uses the most likely outcome of the sampled shots, and runs next to the histogram when the `Medir` button is pressed. The panel shows the **step** and the number of **shots** as two clear blocks at the top, and every expression (the qubit labels, `p(|1⟩)` and the readout correction) is typeset as MathML.

The fidelity of each qubit is taken into account. The probability `p` of a qubit is pulled towards `0.5` by its fidelity `f`:

$$p' = 0.5 + (p - 0.5) \cdot f. \tag{29.1}$$

At `f = 1` the probability is unchanged. At `f = 0` the qubit is fully random. This is the same as a readout error of `(1 - f) / 2`. A noisier qubit therefore gives a flatter distribution.

The JSON contains only the **reduced** (marginal) probability of each qubit, and the marginals cannot give the correlations of an entangled state. Drawing every qubit independently would give a Bell pair `00`, `01`, `10` and `11` with 25 % each, which does not model entanglement at all. The viewer therefore **never** measures by itself: it always asks Python (Qollab, or `make server`), and if there is none, it shows an error instead of a wrong histogram.

The measurement is done by the backend: `sample` draws the outcomes from the statevector and keeps the correlations, so a Bell pair gives only `00` and `11`. See the sampler page. The viewer asks the backend (`HOST.sample` on Qollab, `POST /api/sample` of `make server` locally). The way it is connected is on the viewer on Qollab page.

How the animation works

The vector of each sphere is moved smoothly towards the value of the new step. A small oscillation is added in proportion to the `vibration` value of the qubit, so that a noisy qubit appears to shake.

The fields that are read

The viewer reads only the fields of the contract. The fields that are used are `backend`, `total_qubits`, `initial_fidelity`, and, for each step, `step`, `operation`, `targets`, `phases`, `angle`, `cumulative_fidelity`, `cumulative_cost`, `states`, `probabilities`, `virtual_z`, `inactivity`, `fidelities` and `vibration`.

The files of the viewer

The viewer is a set of small ES modules, so that each file has one purpose. `source/frontend/index.html` holds only the markup and the style; the script starts at `source/frontend/js/main.js`.

File	Purpose
`js/main.js`	The entry point: reads the view options, wires the controls, builds the scene and loads the first file.
`js/settings.js`	The view options: defaults, palettes, profiles and their storage. It imports only `state.js`.
`js/viewpanel.js`	The *View and accessibility* button and its panel (profiles and options), and the interface options (text size, contrast, reduced motion).
`js/motion.js`	How the Bloch vector moves between steps: a rotation along the sphere for single-qubit gates, an arc with a blended length otherwise.
`js/sound.js`	The sound of the gates (Web Audio): laser tones, the entangler swell and the VirtualZ click.
`js/camerapad.js`	The on-screen camera buttons (rotate, tilt, zoom, centre).
`js/config.js`	The constants, the URLs and the host hooks (`window.QVIZ_HOST`).
`js/state.js`	The mutable state of the viewer, in one place.
`js/utils.js`	Small helpers (`showStageMessage`, `escapeHtml`, `setError`).
`js/demo.js`	The small example embedded in the page.
`js/shaders.js`	Loads the eight shader sources (the spheres, the lens, the laser and the phonon bus).
`js/three-addons.js`	Loads the three.js addons, with a fallback.
`js/laser.js`	Loads the laser model, and builds the volumetric beams and their colour.
`js/phonon.js`	Builds the phonon bus, and runs the jiggle and the synchronized halo of a two-qubit gate.
`js/trap.js`	Builds the surface-trap chip (substrate, gold pads and field rings) and lights the active electrodes.
`js/horizontal.js`	Builds the horizontal layer (the global beam cylinder, the guide rail, the side optics) and animates it, including the intro pulse.
`js/measurement.js`	Runs the state-dependent fluorescence animation (readout beam, flares, photons and state badges).
`js/environment.js`	Builds the vacuum chamber and optical table (breadboard, coils, collimators, viewport) and sets the fog and lights.

File	Purpose
`js/scene.js`	The scene, the camera, the render loop and the frame-rate counter.
`js/panels.js`	The panels (probabilities, simulation details, step details), the fidelity and the measurement.
`js/data.js`	Loads a visualization and moves through its steps.
`js/timeline.js`	The circuit timeline strip: builds the wires and gates, and marks the current step. It does not import `data.js` or `controls.js` (the controls wire the click).
`js/selection.js`	The selection marker around the ions of the gate that the pointer is over in the timeline.
`js/controls.js`	The playback bar (step buttons, play, reset, slider) and the drag and drop.

No module imports a module that imports it back, so the dependency graph is easy to follow. A headless test (a stub DOM and a stub `three`) runs the modules without a browser.

30. The viewer on Qollab

This page explains how the viewer of `source/frontend/index.html` runs on Qollab, and how to change it without breaking that. Read it before changing the viewer, the measurement, `qollab_api.py` or `resources/scripts/package_qollab.py`.

At a glance

This section is the short version, to be reused in the README and in the project card.

- **What it is.** QViz shows, step by step, what a trapped-ion computer does with a quantum circuit: each qubit is a Bloch sphere, and every native gate (GPI, GPI2, MS or ZZ, virtual Z) moves it. The fidelity of the circuit drops as the machine accumulates error, and the *Medir* button shows what a measurement would give.
- **How a circuit gets in.** A person loads a circuit written in **OpenQASM 2 or 3** (`.qasm`). The platform reads it, transpiles it, translates it to the native gates of IonQ, simulates it exactly and adds the error model. Only then does it become the JSON that the viewer draws. A gate that cannot be translated to IonQ's native gates (`opaque`, an undefined gate, `reset`, control flow, ...) is rejected with an error that says why.
- **Where it runs.** On **Qollab**, the whole thing runs in the browser: Python (Pyodide) for the backend and a JS project for the viewer, with the backend that the user selects (the built-in simulator or an IonQ simulator). Locally, `make server` serves the viewer and runs the same Python code, so both give the same result. The viewer has no calculation of its own: compiling and measuring always go through Python, because the marginal probabilities in the JSON cannot give the correlations of an entangled state.
- **Cost.** Circuits that a person loads never send the IonQ job (they are transpiled locally), so any number of files can be tried on Qollab, where each run allows one job.
- **License and dependencies.** MIT for our code. Qiskit, qiskit-ionq and qiskit-qasm3-import (Apache-2.0) and NumPy (BSD-3-Clause) are dependencies and do not change that, as the organizers confirmed.

The rule

The viewer is a standalone page: it works without Qollab, served by `python -m http.server`, and it must keep doing so. Everything that Qollab needs is added as an optional hook (see below) or is generated at build time.

What Qollab does and does not allow

These facts come from the documentation of Qollab (`documentation/prompts/ qollab_context.md`), and they decide the design.

Fact	Consequence
The project type is decided by the files of the root: `index.html` and `main.js` make a JS project.	The project that is uploaded needs `index.html` and `main.js` in its root.
`index.html` is body content only, and its `<script>` tags do not run.	The script of the viewer goes to `main.js`.
Only `main.css` is loaded as a style sheet.	The style of the viewer goes to `main.css`.

Fact	Consequence
JS cannot read the files of the project.	The shaders and the laser model are embedded in `main.js`, and the circuits come from Python, not from `out/`.
Python is called from JS with `callPromising`, and a `dict` arrives as a `Map`.	`qollab_api.py` returns text (JSON strings).
`three` (0.186.1) is imported by name, and the import map of the viewer is not read.	The prelude imports `three` and the addons, and the `import` lines of the viewer are removed.
The other files of the project are **not served**: JS cannot `fetch` them, and a relative `import("./js/main.js")` fails with `error loading dynamically imported module`.	The modules of the viewer are bundled into `main.js`.
At most 100 files per project, 2 MB each.	The repository (more than 100 files) is not uploaded as it is.
IonQ backends accept one job per run.	Only the first compilation works on IonQ. Measuring sends no job.
A JS project runs in a sandboxed iframe.	The viewer has its own `body`, so its CSS is copied as it is.

How it is built

```
make qollab
```

The command runs `resources/scripts/package_qollab.py`, which writes the folder `qollab/` (not committed). Its **content** is uploaded to Qollab. The folder has:

File	Where it comes from
`index.html`	The markup of the `<body>` of `source/frontend/index.html`, without the script.
`main.css`	The `<style>` block of `source/frontend/index.html`, unchanged.
`main.js`	A generated prelude (the Qollab services), then all the modules of `source/frontend/js` bundled in one script.
`qollab_api.py`, `requirements.txt`	The files of the root.
`source/backend/`, `resources/examples/`	The libraries.

`main.py` is left out on purpose. In a JS project it could also run, compile a second time and spend a second IonQ job. The viewer does what `main.py` does.

The viewer is copied, not rewritten, so a change in `source/frontend/index.html` or in `source/frontend/js` reaches Qollab with `make qollab` and nothing else. Do not edit the files of `qollab/`.

The prelude of `main.js`

The viewer is modular (`source/frontend/js`), but Qollab serves no file of the project besides the ones it loads itself: a relative `import("./js/main.js")` fails there with `TypeError: error loading dynamically imported module: …/_nuxt/js/main.js`. So `main.js` is **one script**: the prelude, followed by the modules bundled by `bundle_modules` in `resources/scripts/package_qollab.py`. The prelude runs first, so that the host is ready when the modules read it.

1. **The host.** It defines `window.QVIZ_HOST` with the services below. If Python cannot be reached, the message is shown in the `error` line of the viewer, and the viewer falls back to its own behavior. When Python raises an error of QViz (for example, a machine that is not supported), the prelude keeps only its message and drops the traceback, so the `error` line shows a sentence a person can read. Any other error is shown as it is.
2. **The files.** `source/shaders/` (`bloch`, `lens` and `laser`) and `resources/models/laser_lens.obj` and `.mtl` are embedded as strings, because the viewer cannot `fetch` them.
3. **The modules.** Each module of `source/frontend/js` becomes an `async` function with its own scope, in the order of their imports, and returns what it exports; the modules that import it destructure that object. Two modules may therefore declare the same private name. The imports of `three` are kept once at the top. The addons are loaded in `three-addons.js` with a fallback, so that the viewer starts without the laser and the environment light when an addon is not available. The bundler follows the style of the modules: `export` before a declaration, `import {…}` or `import * as x` from `./name.js`; `export default`, `export { … }` and default imports are rejected at build time.

The hooks of the viewer

The viewer reads `window.QVIZ_HOST` once, as `HOST`. Every field is optional. When a field is missing, the viewer does what it always did.

Field	Type	What the viewer does with it
`shaders`	`{vert, frag, lensVert, lensFrag, laserVert, laserFrag}`	Uses these sources, instead of fetching `source/shaders/`.
`laser`	`{obj, mtl}`	Parses this text for the laser model, instead of fetching `resources/models/`.
`circuits`	`[{file, label}]`	Lists them in the selector, instead of reading `out/manifest.json`.
`load`	`async (file) -> object`	Gets the JSON of an entry of the selector, instead of fetching `out/<file>`.
`sample`	`async (data, step, shots) -> [[bitstring, count], …]`	Measures with the backend.
`compileQasm`	`async (text) -> object`	Compiles a circuit written in OpenQASM 2 or 3 (`qollab_api.run_qasm`, without the IonQ job). It is how the user loads a circuit: the button *Cargar circuito* and dropping a `.qasm` file both call it, and the circuit becomes a new entry of the selector. The viewer no longer loads JSON files; with-

Field	Type	What the viewer does with it
		out this hook it asks the local server (`make server`).

The test `test_every_service_of_the_prelude_is_used_by_the_viewer` fails if the prelude defines a service that the viewer does not read. **To add a service:** read it in `source/frontend/index.html` as `HOST.<name>` with a fallback for when it is missing, define it in the prelude of `resources/scripts/package_qollab.py`, add the Python function to `qollab_api.py` if it needs one, and document it in this table.

The measurement

The `Medir` button always draws the outcomes with the backend: `HOST.sample` on Qollab, or `POST /api/sample` of the local server (`make server`), which runs the same function.

1. `HOST.sample(data, step, shots)` sends the JSON text of the current file, the step and the shots to `qollab_api.sample_step`, which calls `backend.sample` (see the sampler).
2. The reply is a list of `[bitstring, count]` pairs, from the most to the least frequent. It is a list and not an object because JavaScript sorts the integer-like keys of an object (`"10"`, `"11"`).
3. The panel is drawn as before, without the note about entanglement, because the measurement keeps the correlations.
4. If the backend fails, or there is no Python, the error is shown. There is no fallback: sampling the marginals independently does not model entanglement.
5. If the user changes the step or the file while Python works, the reply is dropped.

On Qollab, a Bell pair at its last step gives `00` and `11` only, and at step 4 it gives `01` and `10` at about 50 % each.

`qollab_api.py`

Each function is called from JS with `callPromising`, and takes and returns plain values or text.

Function	Meaning
`circuits()`	The example circuits, as JSON text: `[{"file", "label"}]`.
`run(backend, circuit, shots, entangler)`	Compiles an example circuit and returns the JSON text.
`run_qasm(backend, qasm, shots, entangler)`	Compiles a circuit written in OpenQASM 2 or 3, locally (no IonQ job), and returns the JSON text.
`sample_step(visualization, step, shots)`	Measures a step and returns the counts as a JSON list of pairs.

Problems found on Qollab and how they were solved

Each of these is a limit of the platform that broke something. Some were found by running on Qollab, and some are in its documentation. Each one is why a piece of the packaging exists, so keep them in mind before simplifying `resources/scripts/package_qollab.py`.

What failed	Cause	Solution
`import backend` failed with `ImportError` on `native_2q_gate`.	Qollab ships qiskit-ionq 1.0.2, which does not have that helper (it exists from 1.0.3).	The function is written in `provider.py`; the code is tested from 1.0.0 to 1.1.1.
The buttons of the page did nothing, or the JSON was `{}`.	A `dict` returned by Python reaches JS as a `Map`; scripts of the HTML do not run.	`qollab_api.py` returns text (JSON strings); the script lives in `main.js`.
The upload would be rejected.	The repository has more than 100 files, and Qollab refuses a project over that limit.	`make qollab` builds a folder with the files that are needed.
"only one job is allowed per code run" on IonQ.	The compilation sent a second job.	Loaded circuits are transpiled locally (`allow_remote=False`); `main.py` is left out of the JS project.
`TypeError: error loading dynamically imported module: …/_nuxt/js/main.js`.	The prelude loaded the modules of the viewer with a relative `import()`, and Qollab does not serve those files.	`main.js` is one script: the prelude and the bundled modules.
A Python error showed a long traceback in the viewer.	The error crosses to JS with its traceback.	The prelude keeps only the message of the errors of QViz.

How it was verified

On Qollab (Built-in simulator, qiskit 2.5.2, qiskit-ionq 1.0.2, NumPy 2.4.6, with Pyodide v314.0.7) a diagnostic page (since removed; it is in the git history) passed all the checks: the call from a click works after the first run, and compilation of a Bell pair and each measurement took a few milliseconds.

Locally, the generated `main.js` was run with the real Python and a simulated page (jsdom, with no WebGL), and the measurement panel gave the expected results with the backend, without a sampler in the host, and when the backend fails.

Verified in Qollab (2026-10-11): the team ran the current package and everything works: the `three/addons/` imports, the 3D scene, the timeline strip, the view and accessibility panel, measurement and loading circuits.

Rules for future changes

- A change for Qollab goes in a hook or in the prelude, not in the viewer.
- Keep the viewer working alone: with `HOST` empty, it must behave as before.
- Python functions that JS calls return text, never a `dict`, and ordered data is returned as a list.
- Do not call `backend.run` from a function that a click calls. On IonQ it is one job per run.
- After a change, run `make test`, then `make qollab`, upload the folder and run the diagnostic page if anything fails.
- Keep `requirements.txt` equal to `pyproject.toml`, and inside the versions of the installation page.

XI Development

31 Tests . . . 151
31 The shared circuits . . . 151
31 The fixtures . . . 151
31 The test files . . . 152
31 The oracles . . . 153
31 The golden file . . . 153
31 Numerical tolerances . . . 153
31 Continuous integration . . . 154

32 Demos . . . 155
32 The shared helpers . . . 155
32 `demo_transpiler.py` . . . 155
32 `demo_translator.py` . . . 155
32 `demo_simulator.py` . . . 155
32.1 `demo_vectorizer.py` . . . 155
32.1 `demo_metrics.py` . . . 156
32.1 `demo_compile.py` . . . 156
32.1 `demo_adder.py` . . . 156
32.1 `package_qollab.py` . . . 156
32.1 `generate_outputs.py` . . . 156

33 Tooling and workflow . . . 157
33 The Makefile . . . 157
33 The scripts . . . 158
33 Continuous integration . . . 159
33 Branches and commits . . . 159
33 How a module is built . . . 159
33 Working with code agents . . . 160
33 The tests of the project itself . . . 160
33 Before closing a change . . . 160

34 Style and conventions . . . 161
34 The layout of the project . . . 161
34 One module per package . . . 161
34 Imports . . . 161
34 Purity . . . 162
34 No printing inside the library . . . 162
34 Comments and docstrings . . . 162
34 Type hints . . . 162
34 Line length and formatting . . . 162
34 The theme of the book . . . 162
34 The memory of the project . . . 163
34 Git . . . 163

31. Tests

The tests are found in the `source/tests/` directory. They are run with `pytest`:

```
make test
```

One file exists for each module. The name of a file is `test_<module>.py`.

The shared circuits

The file `source/tests/circuits.py` holds the circuits that are used by several test files, together with a helper. Every circuit is built by a function, so that a fresh circuit is made for each test.

Function	Circuit
`single_x()`	One qubit, with an `x` gate.
`single_h()`	One qubit, with a Hadamard gate.
`rotations()`	One qubit, with three arbitrary rotations.
`bell()`	Two qubits, in a Bell state.
`bell_measured()`	The Bell circuit, with a final measurement.
`ghz3()`	Three qubits, in a GHZ state.
`swap_test()`	Three qubits, with a `swap` and a `ccx`, so that decompositions are forced.
`idle_qubit()`	Three qubits, where one qubit is never used.

The helper `make_info(entangler="ms")` builds a synthetic `BackendInfo`. It is used by the tests that do not need a real backend.

The fixtures

The file `source/tests/conftest.py` defines two fixtures. A fixture is a value that `pytest` prepares for a test.

Fixture	Value
`basic_backend`	A `BasicSimulator`, the ideal local backend.
`chip_backend`	A `GenericBackendV2` with 20 qubits, a coupling map and the basis `["cz", "rz", "sx", "x"]`. It is an unsupported backend for `compile`, and it is used to test the layout and the rejection.

No network is used and no credential is needed. Every IonQ backend is mocked.

The test files

File	What is checked
`test_smoke.py`	The public API is exactly the expected one, and a `NativeOp` is immutable.
`test_provider.py`	The provider detection, the entangler rule and the hardware rule.
`test_transpiler.py`	The three IonQ strategies, the fallback and the size limit.
`test_translator.py`	Every rule is checked by an equivalence test. Invariants are also checked, and malformed native payloads are rejected with the index of the entry.
`test_simulator.py`	Hand-computed cases, the statevector oracle, and the gate matrices.
`test_vectorizer.py`	The round trip from Cartesian to spherical and back, and the probabilities.
`test_metrics.py`	Hand-computed values, monotonicity, the virtual Z rule and the ideal profile.
`test_compile.py`	The schema, a golden file, the export, the forced entangler, the argument validation, the schema cap (equal to `MAX_QUBITS`) and the rejection of unsupported backends.
`test_normalize.py`	The ZZ and MS angle normalization is equivalent to the original gate, and the angles stay in range.
`test_formatter.py`	Which fields are present for each operation, the rounding and the negative zero.
`test_exporter.py`	The JSON text, the file output and the rejection of `NaN` and infinities.
`test_examples.py`	The QFT matrix, its inverse, the sum computed by the adder, and the schema of a compiled adder.
`test_performance.py`	The GHZ and layered circuits: the state, the reproducibility with a seed, the argument errors, and the 16-qubit files compiled against the schema.
`test_package_qollab.py`	The Qollab folder fits in the file limit and has the files of a JS project, without `main.py`; the viewer package is the real viewer (markup, style, script and embedded shaders); every service of the prelude is read by the viewer; and the generated `main.js` is valid JavaScript (when Node is installed).

File	What is checked
`test_memory.py`	`memory.md` keeps the sections that a new session needs and stays short, and `CLAUDE.md` and `AGENTS.md` point to it.
`test_requirements.py`	`requirements.txt` equals the dependencies of `pyproject.toml`, and it uses only what Qollab accepts.
`test_qollab_api.py`	The entry point and the functions that Qollab pages call: the `entangler` field, the example circuits, the JSON text that is returned, the order of the counts, and the output of `main`.
`test_sampler.py`	The joint distribution against the marginals of every step and against `Statevector`, the Bell outcomes, the bit order, the noise, the seed, and the rejection of bad steps, shots and data.
`test_server.py`	The local server of the viewer: compiling an OpenQASM text, sampling a step, the error replies (`400`, `404`, `413`), and that the relative URLs of the viewer reach the routes of the server from `/source/frontend/`.
`test_importer.py`	The reading of OpenQASM 2 and 3, and the messages for what cannot be loaded.
`test_tomography.py`	The estimates (with a fixed seed), the single run call and the hardware guard.

The oracles

An oracle is a trusted source of truth that is used for comparison.

- **For the translator:** the circuits `Operator(to_qiskit(native))` and `Operator(circuit)` are compared with the `equiv` method. The `equiv` method ignores the global phase, which has no physical meaning.
- **For the simulator:** the state is compared with `qiskit.quantum_info.Statevector`. A partial trace gives the reduced state of each qubit.
- **For the JSON:** the result is validated with the JSON schema that is stored in `documentation/schema/visualization.schema.json`.

The golden file

The file `source/tests/golden/bell_basic.json` is a snapshot of the full output for the Bell circuit on the `BasicSimulator`. The test compares the new result with this file. When a change to the output is intended, the golden file is regenerated, and the change is mentioned in the review.

Numerical tolerances

The tolerance is `atol = 1e-9` for the statevector comparisons. The tomography is statistical, so a larger tolerance is used there: `5 / sqrt(shots)`.

Continuous integration

The checks are run automatically on GitHub Actions, by the workflow in `.github/workflows/ci.yml`. It runs on every push to `main` and `development`, and on every pull request.

Two jobs are defined.

Job	What it does
Lint	Runs `flake8` and `pydocstyle`, exactly as `make lint` does.
Tests	Runs `pytest`, exactly as `make test` does, on Python 3.12 and 3.13.

Python 3.12 is the oldest supported version, and NumPy also needs 3.12 or newer. A red mark on a pull request means that the style or a test has to be fixed before the change is merged.

32. Demos

The demo scripts are found in the `resources/scripts/` directory. They are meant to be read and run by a person, so they use `print`. They are not part of the library. Each script prints a readable result for one stage of the pipeline.

A script is run with the `M` variable.

```
make demo M=transpiler
make demo M=translator
make demo M=simulator
make demo M=vectorizer
make demo M=metrics
make demo M=compile
make demo M=adder
```

The shared helpers

The file `resources/scripts/_common.py` holds the helpers that the demos share.

Function	Meaning
`make_info(entangler)`	Builds a synthetic `BackendInfo` for the given entangler.
`bell()`	Builds the Bell circuit.
`ghz3()`	Builds the GHZ circuit.

`demo_transpiler.py`

The provider information and the transpiled circuit are printed for each local backend: the `BasicSimulator`, a 20-qubit chip with a coupling map and a fake backend without a target. `compile` accepts only the first one; the other two show how the transpiler alone behaves.

`demo_translator.py`

The native operation list of the Bell and GHZ circuits is printed, for both the MS and the ZZ entangler. The count of each gate type is also printed.

`demo_simulator.py`

A table is printed. There is one row per step and per qubit, and the values `(r, theta, phi)` are given in degrees for the angles.

32.1 `demo_vectorizer.py`

The same table is printed, but with the Cartesian values `(x, y, z)` and with the probability `p(|1⟩)`.

demo_metrics.py

A table is printed with the operation, the accumulated fidelity, the per-qubit fidelities, the inactivity, the vibration and the accumulated cost.

demo_compile.py

The full pipeline is run for the Bell and GHZ circuits. Two files are written: `out/bell.json` and `out/ghz3.json`. The `out` directory is ignored by Git. The files can be opened with the basic viewer.

demo_adder.py

The QFT adder is built and compiled. The Fourier matrix is printed for two qubits, and the adder is run for three input pairs. One JSON file is written for each pair, for example `out/adder_n2_1_1.json`. The example is described on the QFT adder page.

package_qollab.py

This script is run with `make qollab`. The viewer, the way it is built and the rules for changing it are on the viewer on Qollab page.

The whole repository has more files than Qollab accepts (100 per project), and Qollab looks only at the files of the root to decide the type of the project. The script therefore builds the folder `qollab/` (not committed) with a complete **JS project**: `index.html`, `main.css` and `main.js` made from the viewer (its modules bundled), `qollab_api.py`, `requirements.txt` and the libraries `source/backend/` and `resources/examples/`. `main.py` is left out, because in a JS project it could compile a second time and spend a second IonQ job. The content of the folder is uploaded to Qollab. See `documentation/prompts/qollab_pruebas.md`.

generate_outputs.py

This script is run with `make outputs`, not with `make demo`. It compiles a small set of circuits, including two 16-qubit performance circuits, writes the JSON files to `out/`, and writes `out/manifest.json` for the viewer. See the Performance circuits page.

33. Tooling and workflow

This page lists the commands of the project, the automated checks, and the way the team and the code agents work together. Everything is run from the repository root.

The Makefile

`Makefile` is the single entry for the commands of the project. It includes two smaller files: `resources/make/Makefile.git` for the branch workflow and `resources/make/Makefile.quarto` for this book.

Setting up and checking

Command	What it does
`make venv`	Creates `.venv`.
`make install`	Installs the package in editable mode with the development extras (`pytest`, `jsonschema`, `flake8`, `pydocstyle`).
`make install-requirements`	Installs `requirements.txt`, the list that Qollab reads.
`make test`	Runs the whole suite with `pytest source/tests/ -v`.
`make lint`	Runs `flake8` and `pydocstyle` on the library, the examples, the adapters, the scripts and the tests.
`make demo M=<module>`	Runs `resources/scripts/demo_<module>.py`.

Producing and serving

Command	What it does
`make outputs`	Writes the example JSON files and `manifest.json` to `out/`.
`make qollab`	Builds `qollab/`, the JS project to upload to Qollab.
`make server`	Serves the viewer at `http://localhost:8000` with compiling and measuring in Python (`PORT=9000` changes the port).
`make open`	Like `server`, and also opens the browser.

Command	What it does
`make resume`	Shows the branch, the last commits, the unpushed changes and the pending list of `memory.md`.

Diagrams

Command	What it does
`make install-plantuml`	Installs PlantUML and Graphviz with `apt`.
`make render-puml`	Renders `documentation/diagrams/*.puml` to `documentation/diagrams/svg/`.

The SVG files are committed, so building the book does not need PlantUML.

The book

Command	What it does
`make render`	Renders every format configured in `documentation/_quarto.yml`.
`make html`	Renders the HTML book only.
`make pdf-typst`	Renders the PDF with Typst.
`make preview`	Serves the book on port 4321 and re-renders a chapter when it is saved.
`make check`	Verifies the Quarto toolchain.
`make clean`	Removes `documentation/_book` and the caches.

The scripts

The scripts in `resources/scripts/` are tools, not part of the library. They may use `print`.

Script	Purpose
`demo_<module>.py`	One per stage (`transpiler`, `translator`, `simulator`, `vectorizer`, `metrics`, `compile`, `adder`). It prints a result that a person can read.
`generate_outputs.py`	Compiles the example circuits and writes `out/`.
`package_qollab.py`	Builds `qollab/`.
`server.py`	The local server of the viewer.
`_common.py`	Helpers shared by the demos.

Continuous integration

`.github/workflows/ci.yml` has two jobs, on every push to `main` and `development` and on every pull request.

Job	Steps
Lint	`flake8` and `pydocstyle`, with the same arguments as `make lint`.
Tests	`pip install -e ".[dev]"`, then `pytest`, on Python 3.12 and 3.13.

A new run of the same branch cancels the previous one. The token has read-only permissions. Because the workflow repeats the Makefile commands, a change that passes `make test` and `make lint` locally passes in CI.

Branches and commits

`Makefile.git` encodes the branch workflow, so that nobody has to remember the commands.

Branch	Purpose	Created by
`main`	Stable code.	`make git-init`
`development`	Integration of the next version. Work is pushed here.	`make git-init`, `make development`
`feature/<name>`	One module or feature.	`make feature NAME=<name>`
`release/<version>`	Stabilising a version.	`make release VERSION=<x.y.z>`
`hotfix/<version>`	An urgent fix of the published code.	`make hotfix VERSION=<x.y.z>`

A feature is merged back with `make finish-feature NAME=<name>`, which uses `--no-ff` so that the branch stays visible in the history. A release or a hotfix is tagged `v<version>` and merged into both `main` and `development`.

The same file has short helpers for daily use: `status`, `log`, `diff`, `stage`, `unstage`, `commit MESSAGE="…"`, `amend`, `push`, `pull` (fast-forward only) and `fetch`.

Commit messages are in English. The first line is a short summary, and a body with bullets is added when it helps.

How a module is built

The specification defines the modules M0 to M10. For each one the same cycle is followed.

1. Check in the status table of the specification that the previous module is validated.
2. Read the complete section of the module: objective, rules, tests and demo.
3. Implement that module only, with its tests in `source/tests/test_<module>.py` and its demo in `resources/scripts/demo_<module>.py`.
4. Run `make test` and the demo. Both must pass.
5. Mark the module *ready to validate* in the specification.
6. Stop and report what was done, how to validate it (with exact commands and the expected output), what is still unverified on Qollab and any deviation from the specification.
7. Continue with the next module when the team agrees.

If the work shows that the specification is wrong, the code and the specification are corrected together and the report says so. A change to the JSON contract is not made without asking.

Working with code agents

Three files in the root tell an agent how to work, and they are written in Spanish because the team is.

File	Content
`AGENTS.md`	The rules: architecture, Qollab restrictions, tests, commands, Git and the scope.
`CLAUDE.md`	Imports `AGENTS.md` and `memory.md`, so that both are loaded at the start of a session.
`memory.md`	The hand-over: current state, decisions, pending items and the history. It is updated after each change and a test keeps it under 150 lines.

At the start of a session the agent pulls, reads `memory.md` and runs `make resume`. At the end it runs `make test` and `make lint`, brings `memory.md` up to date and, only if the team asks, commits and pushes.

The tests of the project itself

Some tests do not check behaviour. They check that the project stays coherent.

Test	Keeps in line
`test_requirements.py`	`requirements.txt` and the dependencies of `pyproject.toml`.
`test_memory.py`	`memory.md`, which must have its sections and stay short.
`test_package_qollab.py`	The Qollab package, its limits and its services.
`test_smoke.py`	The public API of `backend`.

Before closing a change

A short list that applies to any change.

- ☐ `make test` is green.
- ☐ `make lint` is green.
- ☐ If the viewer, `qollab_api.py` or the packager changed: `make qollab`.
- ☐ The affected chapter of this book is updated.
- ☐ `memory.md` has the new state and a line of history.
- ☐ If the JSON contract changed, the schema, the golden file and JSON contract agree, and the team knew.

34. Style and conventions

This page describes the rules that are followed by the code. The rules are checked by two tools, `flake8` and `pydocstyle`, which are run with `make lint`.

The layout of the project

The folders and files of the repository, and what belongs in each, are on Project structure. In short, `source/` holds the library, the viewer, the shaders and the tests; `resources/` holds the examples, the scripts and the makefiles; and `documentation/` holds the specification and this book.

One module per package

Each pipeline stage lives in its own package: `source/backend/core/<module>/<module>.py`. The public function has the same name as the module, and it is a verb.

Module	Function
`transpiler`	`transpile`
`translator`	`translate`, `translate_circuit`, `translate_native_ops`
`simulator`	`simulate`
`vectorizer`	`vectorize`
`metrics`	`measure`
`formatter`	`format`
`exporter`	`export`
`sampler`	`sample`
`provider`	`detect`
`importer`	`load_circuit`
`compiler`	`compile`

Imports

Imports are absolute, from the root of the package.

```
from backend.core.transpiler import transpiler   # correct
from core.transpiler import transpiler            # incorrect
```

The second form does not work in Qollab, because the project root is the first entry of `sys.path` there.

Purity

The public functions are pure. An object is received, and a new object is returned. The input is never changed. There is no global state.

A frozen dataclass is used for every object that travels between the stages, so that an accidental change raises an error.

No printing inside the library

The `print` function is used only in `resources/scripts/` and in `main.py`. Inside `source/backend/core`, the `logging` module is used instead.

```
import logging

logger = logging.getLogger(__name__)
logger.warning("a useful message")
```

Comments and docstrings

- The code is written in English. The identifiers, the comments and the docstrings are in English.
- Every module, every public class and every public function has a docstring.
- A docstring starts with a short line in the imperative mood, for example “Return the …”. The line ends with a full stop. A longer description follows after a blank line when it is needed.
- The units of every parameter and of every return value are stated.

Type hints

Type hints are given in every public signature. The `from __future__ import annotations` line is used at the top of a module, so that the newer syntax works on older interpreters.

Line length and formatting

- The maximum line length is 88 characters.
- The rules `E203` and `W503` are ignored, because they conflict with common formatting tools.
- The settings are stored in `.flake8`. The rule `E501` is ignored for `main.py` only, because Qollab needs the whole signature of `main` on one line to build its form.

The theme of the book

The book has a monochromatic theme: a single grey family, with no second hue. Headings, links, callouts and code all read the same ink, and the active entry of the sidebar is the one that is set in bold.

The theme is split in three SCSS files under `documentation/`:

- `theme-light.scss` is loaded after the built-in `cosmo` theme.
- `theme-dark.scss` is loaded after the built-in `darkly` theme.
- `theme-rules.scss` holds the rules that both variants share. Only the colour tokens change between them.

The colour blocks use the `/*-- scss:defaults --*/` and `/*-- scss:rules --*/` markers that Quarto expects. The syntax highlighting uses Pandoc’s `monochrome` style, with a small grey hierarchy added by `theme-rules.scss`.

The PDF is built with Typst through the `orange-book` extension (`document-class: book`). One default of that extension is tuned in `_quarto.yml`: the part number is made smaller, so a wide numeral such as `VIII` does not run into the part title. The mini table of contents of a part divider page is kept at its default

depth, which lists the chapters of the part and their sections. For it to fit, the backend pages are grouped into several parts (foundations, input and translation, simulation and output, orchestration and extras) instead of one part of fourteen chapters.

The memory of the project

The chat of an agent is emptied between sessions, but the repository is not. The file `memory.md` in the root is the hand-over: the current state, what was decided, what is pending, and how to continue. `CLAUDE.md` imports it, and `AGENTS.md` asks every agent to read it at the start and to update it at the end of each change. `make resume` shows the branch, the last commits, the changes that are not pushed and the pending list. The file is kept short (a test checks that it stays under 150 lines), and the design stays in `documentation/prompts/spec.md`, which wins when the two disagree.

Git

- `main` is the stable branch. `development` is the integration branch.
- One branch is used per module: `feature/<module>`.
- A commit is made only when the team asks for it.
- A commit message is written in English. The first line is a short summary, and a body with bullets is added when it helps.